

INDITEX UDC

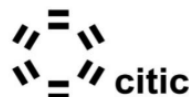
Cátedra de IA en Algoritmos Verdes

MICROCREDENCIAL ALGORITMOS VERDES PARA LA IA: DISEÑO E IMPLEMENTACIÓN

Tema: AI, Graphics & Visual Computing: Usos de la IA para la Reconstrucción 3D. Generación de Datos Sintéticos.

Sesión: Lunes 6 de octubre de 2025

Ponente: José A. Iglesias Guitián



UNIVERSIDADE DA CORUÑA



Financiado por la
Unión Europea
NextGenerationEU



GOBIERNO
DE ESPAÑA
MINISTERIO
PARA LA TRANSFORMACION DIGITAL
Y DE LA FUNCIÓN PÚBLICA



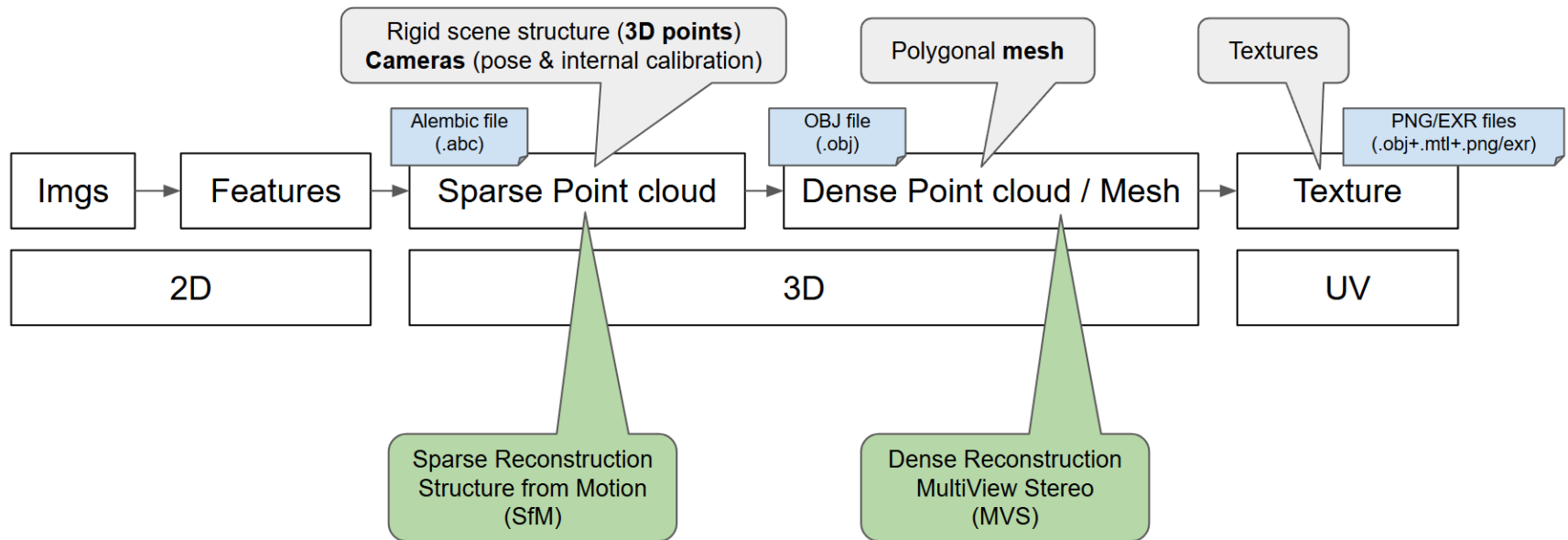
Session Outline

1. **Photogrammetry and Computer Vision Fundamentals**
2. A gentle introduction to Novel View Synthesis (NVS)
3. Synthetic image data generation for AI consumption

Photogrammetry

Photogrammetry Pipeline

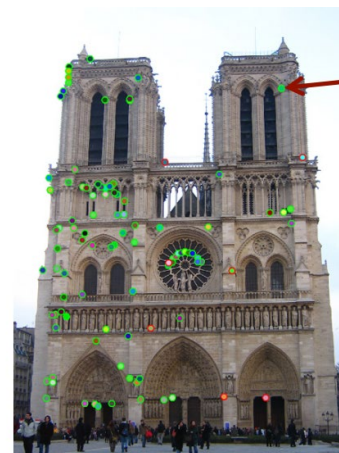
- **Photogrammetry** is the science or technique required to obtain reliable information on the natural environment or physical objects by recording, measuring, and interpreting photographic images.
- A modern photogrammetry reconstruction pipeline may look like this ...



Source: Javier Taibo @ Course on Modeling at UDC (Chapter 10. Photogrammetry)

Photogrammetry Fundamentals

- Camera **extrinsics**:
 - Position
 - Orientation
- Camera **intrinsics**:
 - Sensor size
 - Lens/Optics: focal distance, field-of-view, lens distortion
- A **feature** is a (locally) distinctive and repeatable image pattern (e.g., a point, corner, or small region) that can be reliably detected and described across different viewpoints, scales, or illumination conditions.
 - A **keypoint** identifies **where** the feature is (location)
 - A (feature) **descriptor** represents how to **describe** the local neighborhood represented by a small group of surrounding pixels.



keypoint

descriptor at
the keypoint

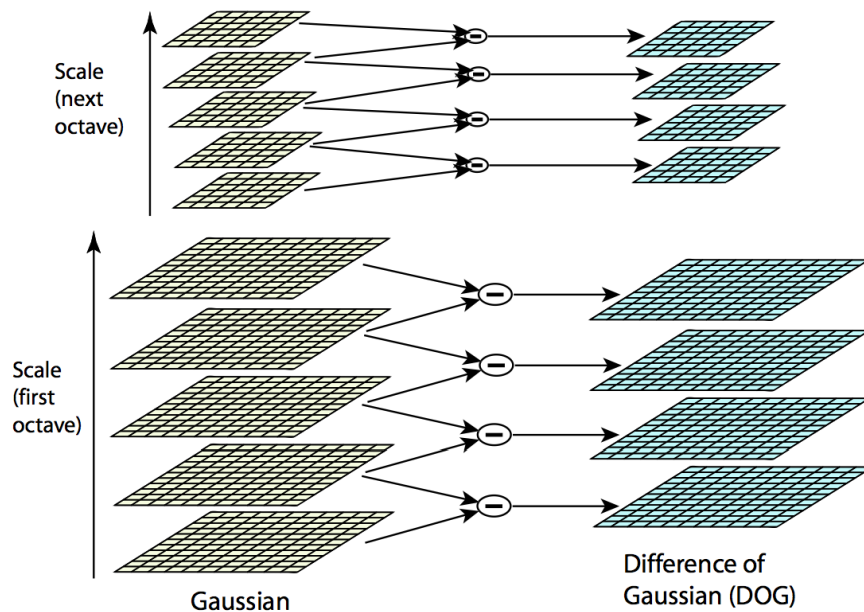
$$f = \begin{bmatrix} 0.02 \\ 0.04 \\ 0.1 \\ 0.03 \\ 0 \\ \dots \end{bmatrix}$$

Source: Cyrill Stachniss. Visual Features (Photogrammetry & Robotics Lab)

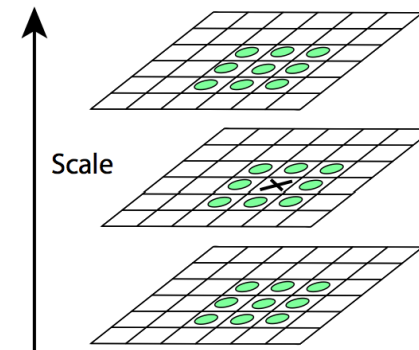
Photogrammetry. Keypoints (DoG)

- Difference of Gaussians (DoG) Over Scale-Space Pyramid

differently sized images



differently blurred images



Source: Cyrill Stachniss. Visual Features (Photogrammetry & Robotics Lab) adapted from [Lowe10]

Photogrammetry. Feature Descriptors

- HOG: Histogram of Oriented Gradients
- **SIFT: Scale Invariant Feature Transform**
- SURF: Speeded-Up Robust Features
- GLOH: Gradient Location and Orientation Histogram
- BRIEF: Binary Robust Independent Elementary Features
- ORB: Oriented FAST and rotated BRIEF
- BRISK: Binary Robust Invariant Scalable Keypoints
- FREAK: Fast REtinA Keypoint
- ...

Source: Cyrill Stachniss. Visual Features (Photogrammetry & Robotics Lab) adapted from [Lowe10]

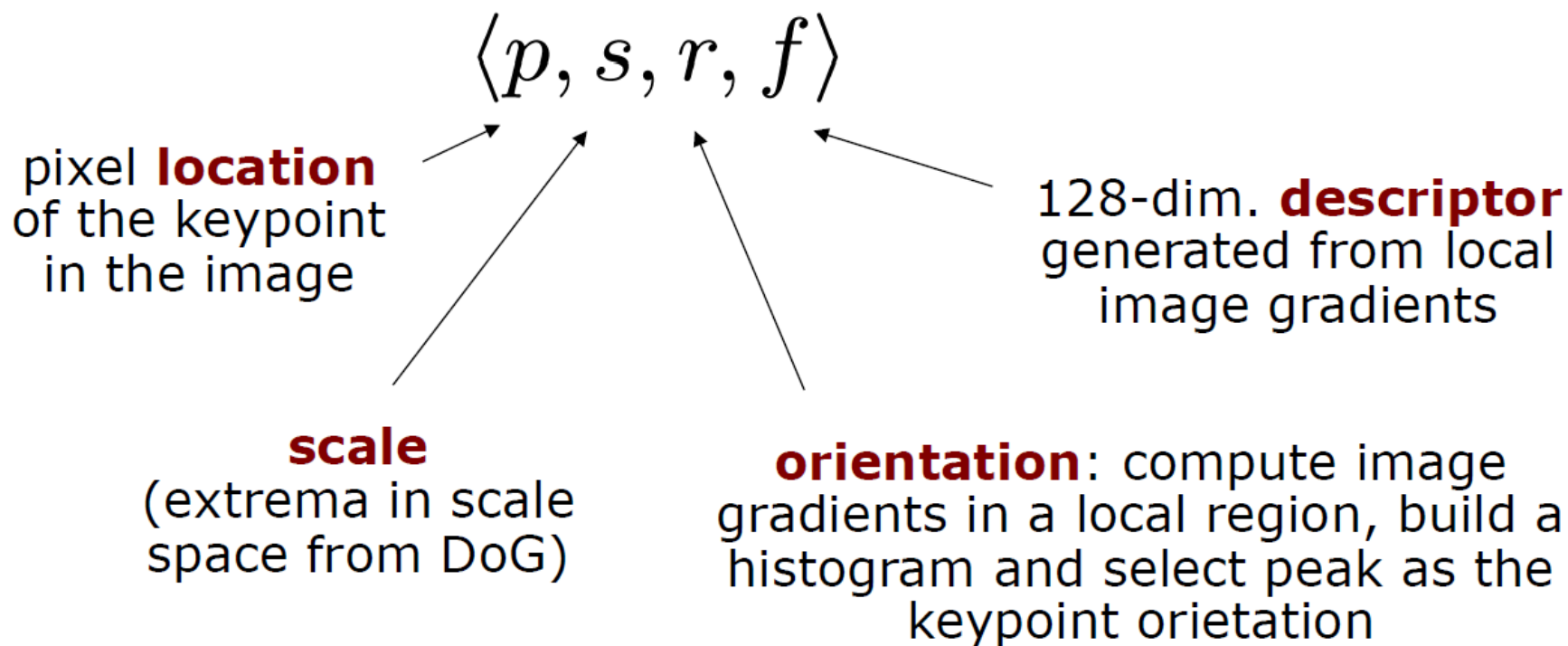
Photogrammetry. SIFT Descriptor

- Image content is transformed into features that are invariant to
 - image translation,
 - rotation, and
 - scale
- They are partially invariant to
 - illumination changes and
 - affine transformations and 3D projections
- Suitable for detecting visual landmarks
 - from different angles and distances
 - with a different illumination

Source: Cyrill Stachniss. Visual Features (Photogrammetry & Robotics Lab) adapted from [Lowe10]

Photogrammetry. SIFT Descriptor

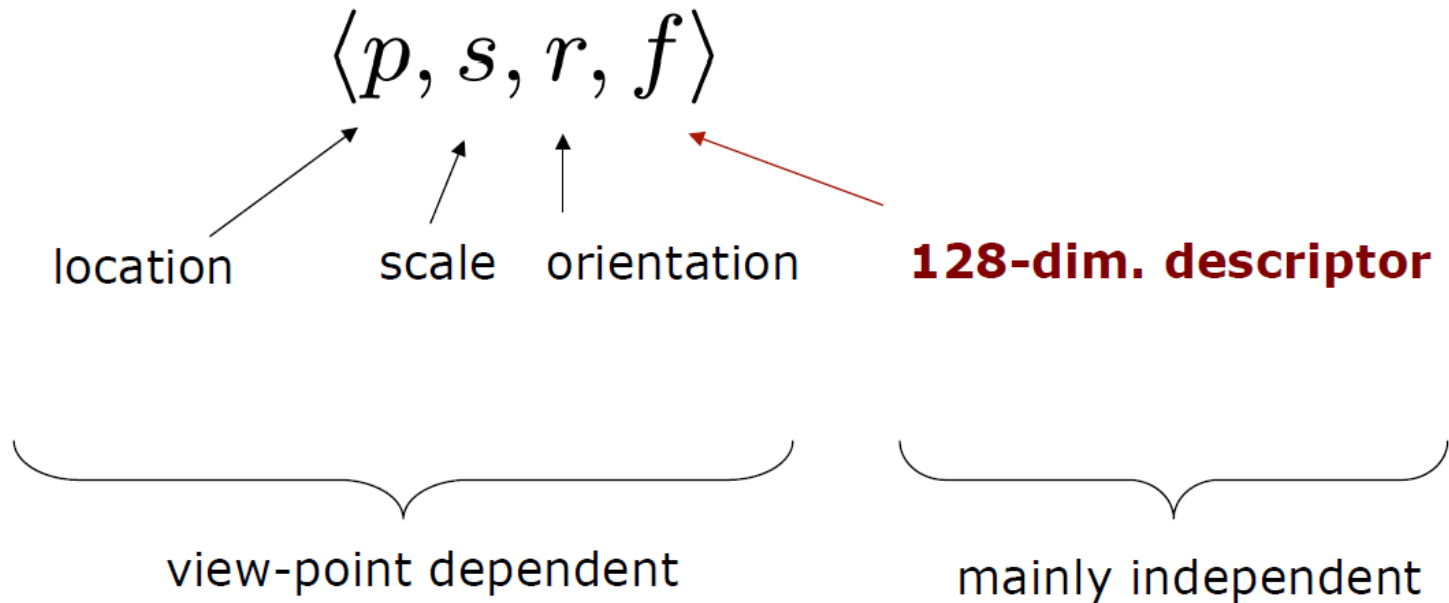
- A SIFT feature is given by a vector computed at a extreme in the scale space



Source: Cyrill Stachniss. Visual Features (Photogrammetry & Robotics Lab) adapted from [Lowe10]

Photogrammetry. SIFT Descriptor

- A SIFT feature is given by a vector computed at a extreme in the scale space



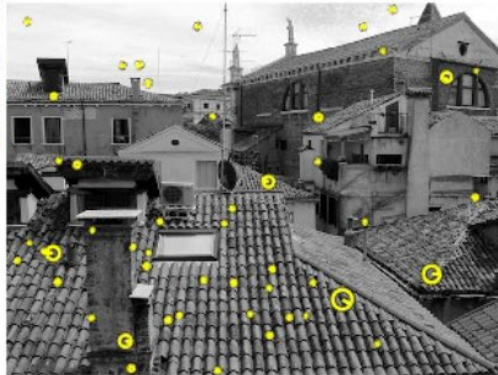
Source: Cyrill Stachniss. Visual Features (Photogrammetry & Robotics Lab) adapted from [Lowe10]

Photogrammetry. SIFT Descriptor

- A SIFT feature is given by a vector computed at a extreme in the scale space



input



key points



descriptor (regions)

Source: Image courtesy of Vedaldi and Fulkerson. Adapted by Cyrill Stachniss. Visual Features (Photogrammetry & Robotics Lab)

Photogrammetry. SIFT Descriptor

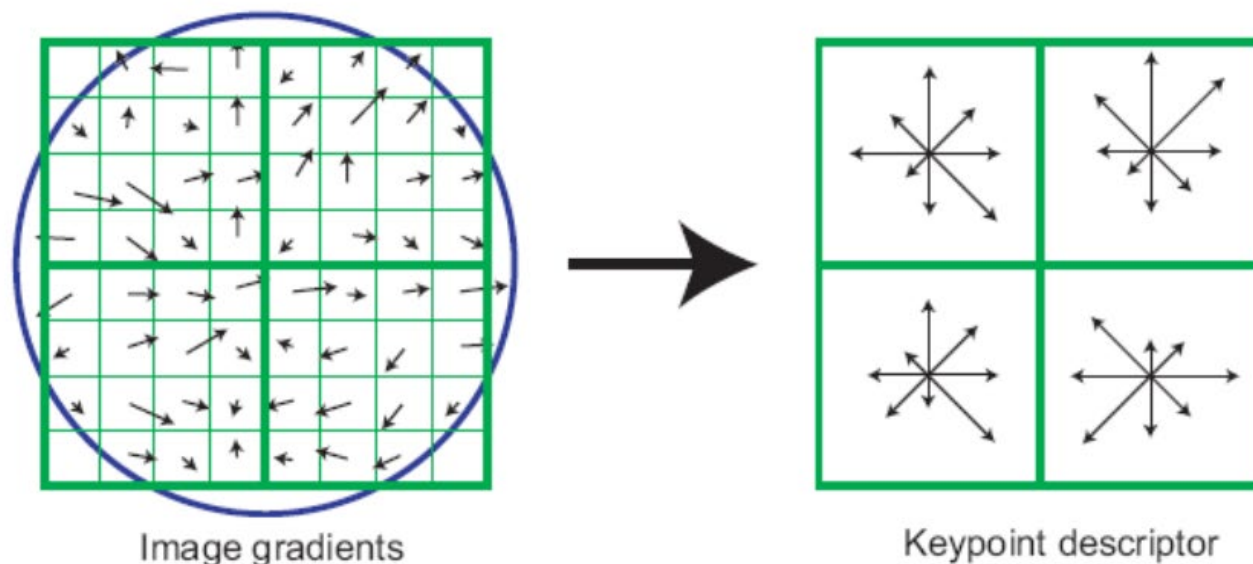
- A SIFT feature is given by a vector computed at a local extreme point in the scale space
- Once a keypoint is found at position (x, y) , scale s , and orientation θ ...
- **Patch extraction:**
 - Take a local neighborhood around the keypoint, scaled according to s .
- **Gradient computation:**
 - For each pixel in the patch, compute gradient magnitude and orientation.
- **Spatial binning:**
 - The patch is divided into a 4×4 grid of cells.
 - Each cell builds an orientation histogram with 8 bins.
- **Concatenation:**
 - 4×4 cells $\times$ 8 bins = **128-dimensional descriptor vector**.
- **Normalization:**
 - The vector is normalized to reduce illumination sensitivity.

Source: Cyrill Stachniss. Visual Features (Photogrammetry & Robotics Lab) adapted from [Lowe10]

Photogrammetry. SIFT Descriptor

- Compute image gradients in local 16x16 area at the selected scale s
- Create an array of orientation histograms (128 dimensions)

Example using a 8x8 area:



Source: Cyrill Stachniss. Visual Features (Photogrammetry & Robotics Lab) adapted from [Lowe04]

Photogrammetry. SIFT (Example Illustration 1/4)

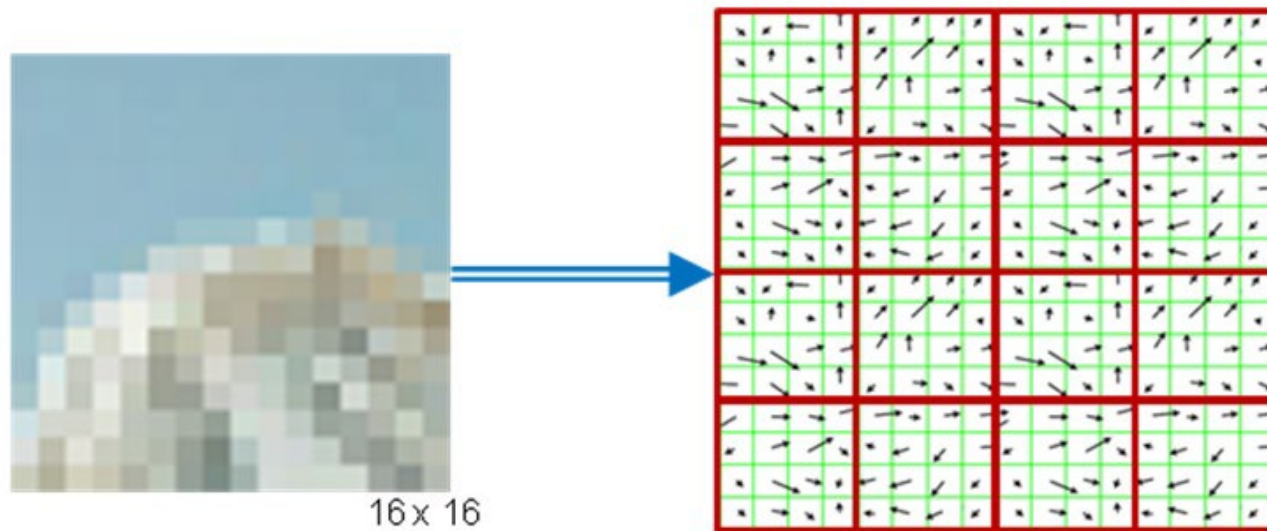
- For a given keypoint, warp the region around it to orientation and scale and resize the region to 16X16 pixels.



Source: Cyrill Stachniss. Visual Features (Photogrammetry & Robotics Lab) adapted from [Lowe04]

Photogrammetry. SIFT (Example Illustration 2/4)

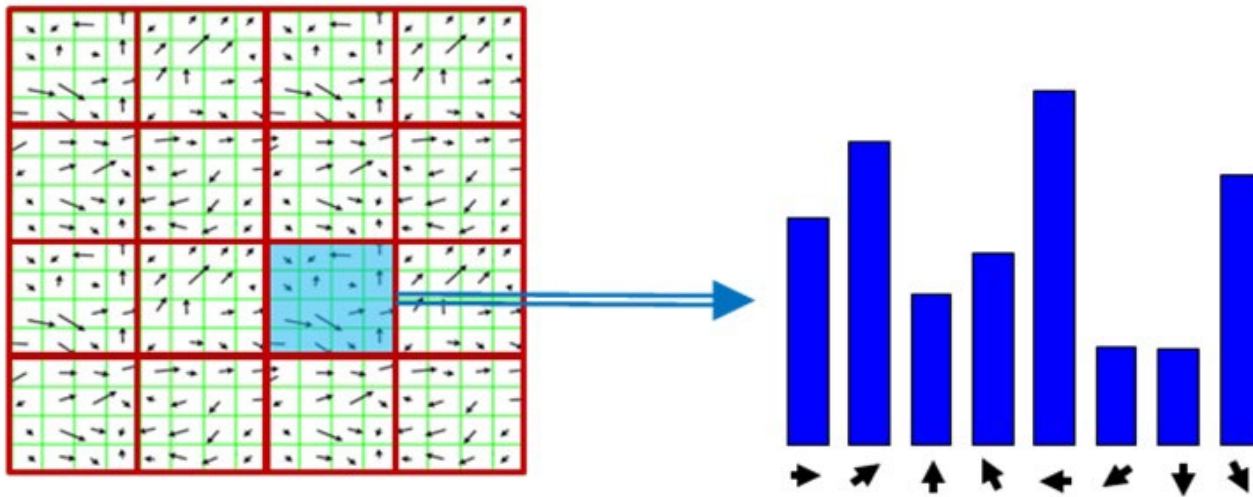
- Compute the gradients (orientation and magnitude) for each pixel, and divide the region into 16 4x4 pixels subregions (highlighted in red)...



Source: Cyrill Stachniss. Visual Features (Photogrammetry & Robotics Lab) adapted from [Lowe04]

Photogrammetry. SIFT (Example Illustration 3/4)

- For each subregion compute gradient direction histogram over 8 directions ...
- The orientation $\theta(x,y)$ decides which bin of the orientation histogram the gradient contributes to:
 - Example: if $\theta = 40^\circ$, it contributes to the bin around 45° .
- The magnitude $m(x,y)$ is the weight of that contribution.
 - Stronger gradients (edges, corners) count more.
 - Weak gradients contribute less, so noisy or flat regions do not dominate.



Source: Cyrill Stachniss. Visual Features (Photogrammetry & Robotics Lab) adapted from [Lowe04]

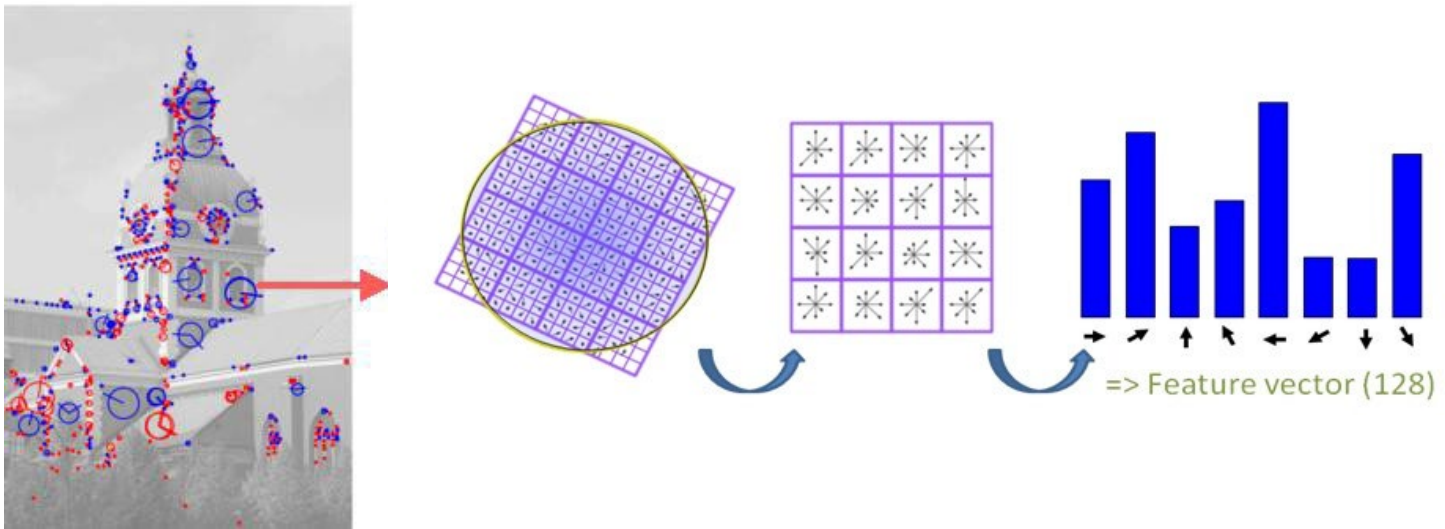
Photogrammetry. SIFT (Example Illustration 4/4)

- Concatenate the histograms to obtain a 128 (16×8) dimensional feature vector



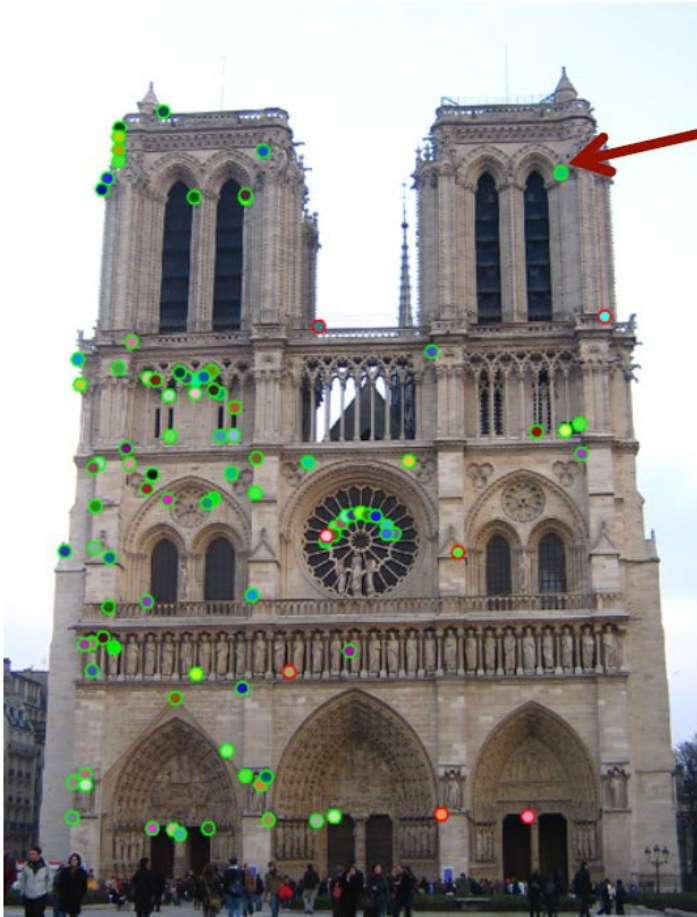
Source: Cyrill Stachniss. Visual Features (Photogrammetry & Robotics Lab) adapted from [Lowe04]

Photogrammetry. SIFT (Recap)



Source: Cyrill Stachniss. Visual Features (Photogrammetry & Robotics Lab) adapted from [Lowe04]

Photogrammetry. Keypoint & Feature Descriptor (Recap)



keypoint
(via DoG)

descriptor (via
gradient histogram)

$$f = \begin{bmatrix} 0.02 \\ 0.04 \\ 0.1 \\ 0.03 \\ 0 \\ \dots \end{bmatrix}$$

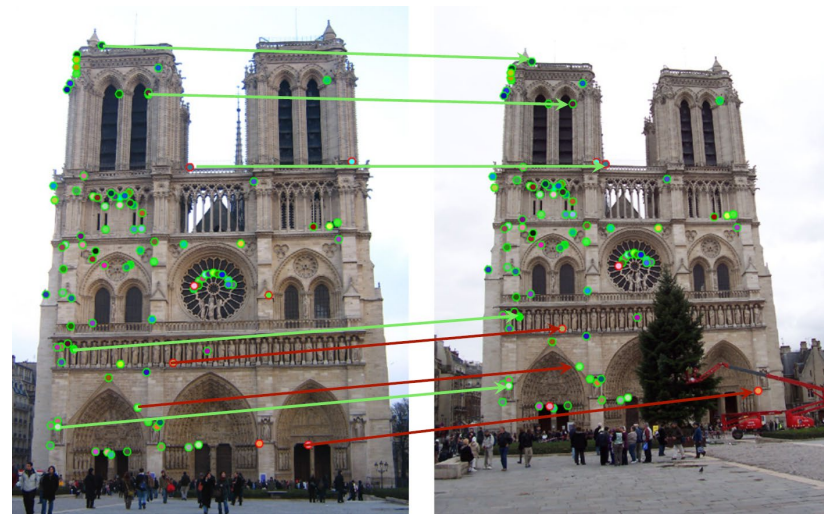
Source: Cyrill Stachniss. Visual Features (Photogrammetry & Robotics Lab)

Photogrammetry. Feature-based matching of keypoints

- Based on descriptor difference
- Lowe's Ratio Test
 - Query a feature q
 - Find the two closest descriptors ()based on Euclidean distance d
 - Test if distance is smaller than a threshold
$$d(q, p_1) < T$$
 - Accept match only if the best match is substantially better than the second:

$$\frac{d(q, p_1)}{d(q, p_2)} < \frac{1}{2}$$

- Outliers still require extra treatment



Source: Cyrill Stachniss. Visual Features (Photogrammetry & Robotics Lab)

Binary Descriptors

Why Binary Descriptors?

- Complex features such as SIFT work well and are a gold standard
- SIFT is expensive to compute
- SIFT has patenting issues
- Binary descriptors aim at generating **small binary strings** that are **easy to compute and compare**

Source: Cyrill Stachniss. Visual Features (Photogrammetry & Robotics Lab)

Binary Descriptors. Key Idea

- **Summary:** binary descriptors encode the **relative intensity of pixel pairs** in a local patch around the keypoint.
- Important **remark:**
 - Use the same pairs.
 - Maintain the same order in which pairs are tested!

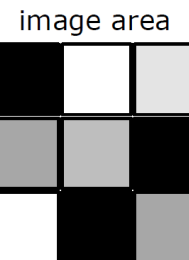
Procedure:

- Select a patch around a keypoint
- Select a set of pixel pairs in that patch
- For each pair, compare the intensities

$$b = \begin{cases} 1 & \text{if } I(s_1) < I(s_2) \\ 0 & \text{otherwise} \end{cases}$$

- Concatenate all b 's to a bit string

Example:



index (for pairs)

1	2	3
4	5	6
7	8	9

pairs: $\{(5, 1), (5, 9), (4, 6), (8, 2), (3, 7)\}$

tests: $b = 0 \quad b = 0 \quad b = 0 \quad b = 1 \quad b = 1$

result: $B = 00011$

Source: Cyrill Stachniss. Visual Features (Photogrammetry & Robotics Lab)

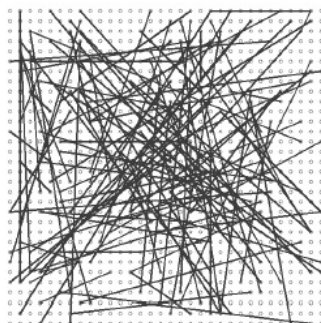
Binary Descriptors. Key Advantages

- **Compact descriptor**
 - The number of pairs gives the length in bits
- **Fast to compute**
 - Simply intensity value comparisons
- **Trivial and fast to compare**
 - Hamming distance

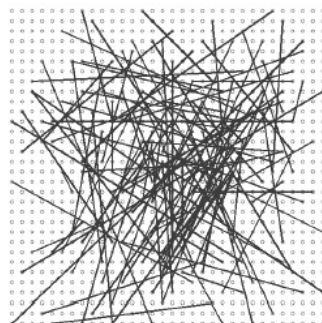
$$d_{\text{Hamming}}(B_1, B_2) = \text{sum}(\text{xor}(B_1, B_2))$$

BRIEF (Binary robust independent elementary features)

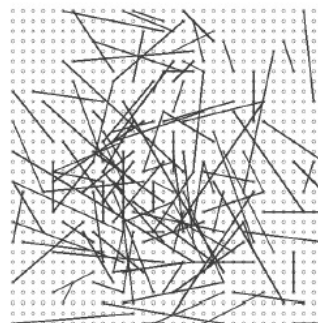
- First binary image descriptor (proposed in 2010)
- 256-bit descriptor
- Provides five different geometries as sampling strategies



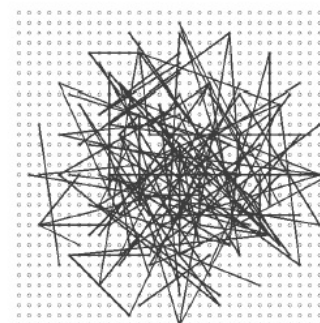
G I
Uniform random sampling



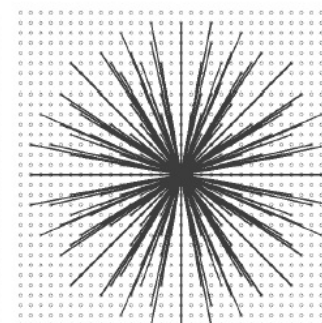
G II
Gaussian sampling



G III
s1 Gaussian; s2 Gaussian
centered around s1



G IV
Discrete location from a
coarse polar grid



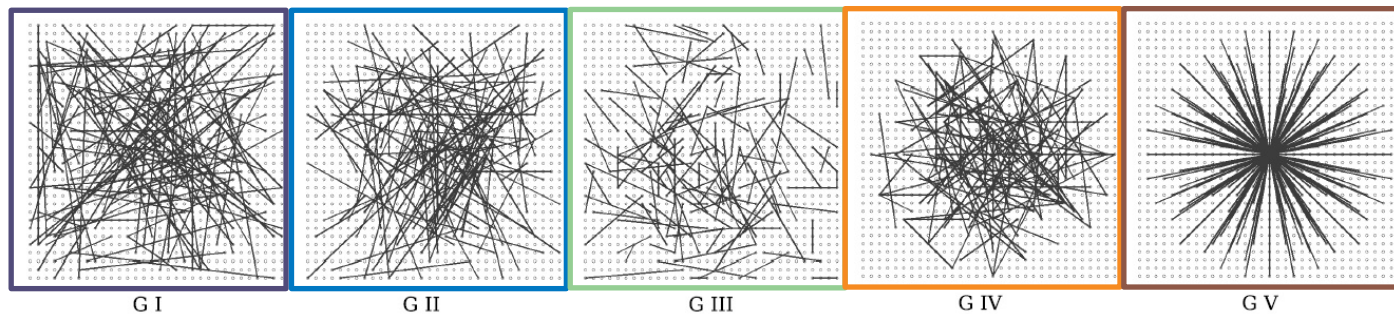
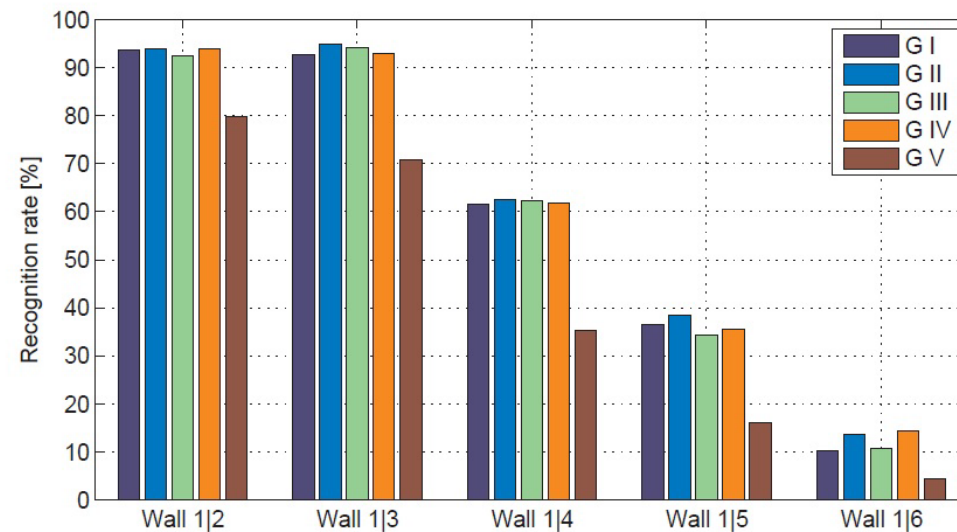
G V
 $S_1 = (0,0)$; s_2 are all locations
from the coarse polar grid

- Noise: operations performed on a **smoothed** image to deal with noise

Source: Cyrill Stachniss. Visual Features (Photogrammetry & Robotics Lab)

BRIEF (Binary robust independent elementary features)

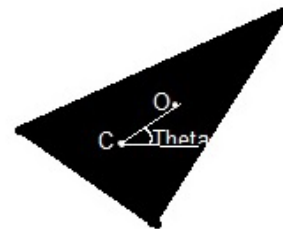
- Performance of the various sampling strategies



Source: Cyrill Stachniss. Visual Features (Photogrammetry & Robotics Lab)

ORB (Oriented FAST Rotated BRIEF)

- An extension of BRIEF that ...
 - Adds rotation compensation.
 - Learns the optimal sampling pairs.



- ORB: Rotation Compensation

- Estimates the center of mass and the main orientation of the area / patch.

- Image moment:
$$m_{pq} = \sum_{x,y} x^p y^q I(x, y)$$

- Center of mass:
$$C = \left(\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}} \right)$$

- Orientation:
$$\theta = \text{atan2}(m_{01}, m_{10})$$

- Given the center of mass and orientation rotate all pair coords: $s' = T(C, \theta) s$
- Use the transformed pixel coordinates for performing the matching test.

Source: Cyrill Stachniss. Visual Features (Photogrammetry & Robotics Lab)

ORB (Oriented FAST Rotated BRIEF)

- Learning Sampling Pairs.
- Pairs should be/have ...
 - **uncorrelated**, so that each new pair adds new information to the descriptor.
 - **high variance**, as it makes a feature more discriminative.
- ORB defines a strategy to select 256 pairs optimizing for both properties above using a training database.

ORB Vs. SIFT

- ORB is 100x faster than SIFT
- ORB: 256-bit vs. SIFT: 4096-bit
- ORB is not scale invariant (achievable via an image pyramid)
- ORB mainly in-plane rotation invariant
- ORB has a similar matching performance as SIFT (w/o scale)
- Several modern online systems (e.g. SLAM) use binary features

Source: Cyrill Stachniss. Visual Features (Photogrammetry & Robotics Lab)

Green AI in Photogrammetry

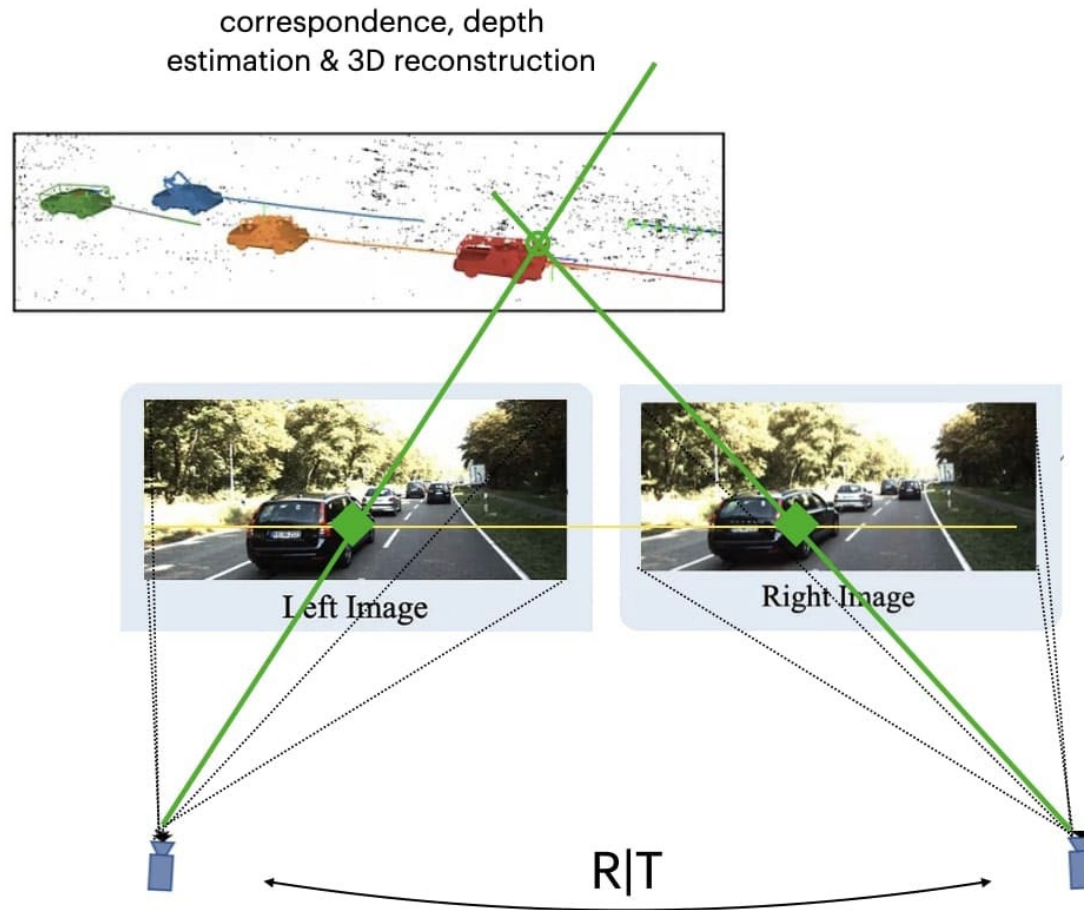
-  Keypoint detection efficiency matters
-  Descriptor optimization saves energy
-  Trade-off between richness and efficiency
-  Parallelization and GPU efficiency
-  Sparse vs. dense features
-  Impact on photogrammetry performance

Green AI in Photogrammetry

-  **Keypoint detection efficiency matters:** Algorithms that detect fewer but more distinctive keypoints reduce unnecessary computation and memory use, lowering energy cost without losing accuracy.
-  **Descriptor optimization saves energy:** Compact descriptors (e.g., binary descriptors like ORB or BRISK) require less computation and storage compared to heavy descriptors like SIFT, cutting CPU/GPU cycles and power use.
-  **Trade-off between richness and efficiency:** While large descriptors (e.g., 128-D SIFT) are powerful, optimized alternatives (e.g., 32- or 64-D learned descriptors) achieve similar performance with less energy demand.
-  **Parallelization and GPU efficiency:** Modern photogrammetry pipelines leverage optimized GPU kernels for descriptor computation, reducing runtime
 - Modern CPUs/GPUs keep consuming **baseline power** even when underutilized ...
 - ... the device is powered but not doing useful work.
 - ➔ **Less idle time = less energy wasted**
-  **Sparse vs. dense features:** Using adaptive feature selection (focusing only on informative regions) prevents overloading the pipeline with redundant descriptors, improving both energy efficiency and reconstruction quality.
-  **Impact on photogrammetry performance:**
 - Faster, lighter keypoint/descriptor computation = quicker Structure-from-Motion (SfM) initialization.
 - Less storage and bandwidth = more scalable Multi-View Stereo (MVS).
 - Overall ➔ shorter processing time, lower energy footprint, and more sustainable 3D reconstruction workflows.

Stereo Vision

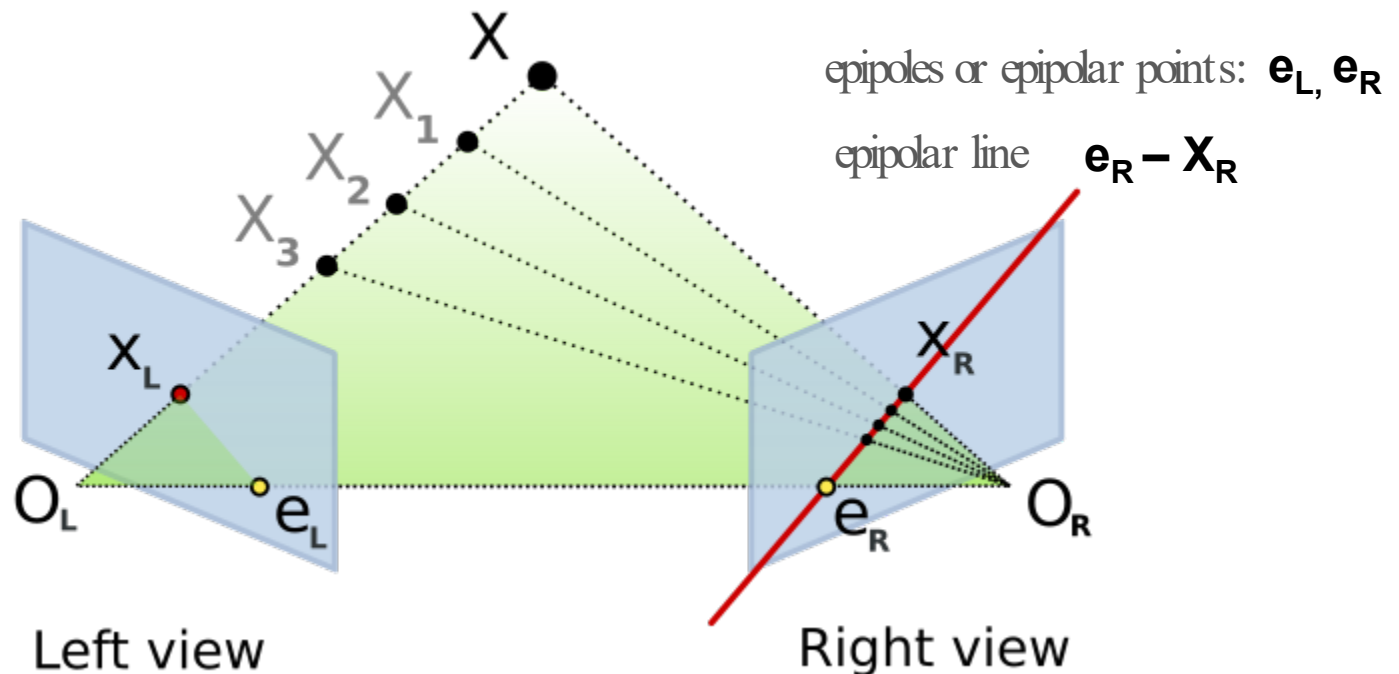
Stereo Vision. Motivation



Source: <https://pyimagesearch.com/2024/10/14/photogrammetry-explained-from-multi-view-stereo-to-structure-from-motion/>

Stereo Vision. Epipolar Geometry

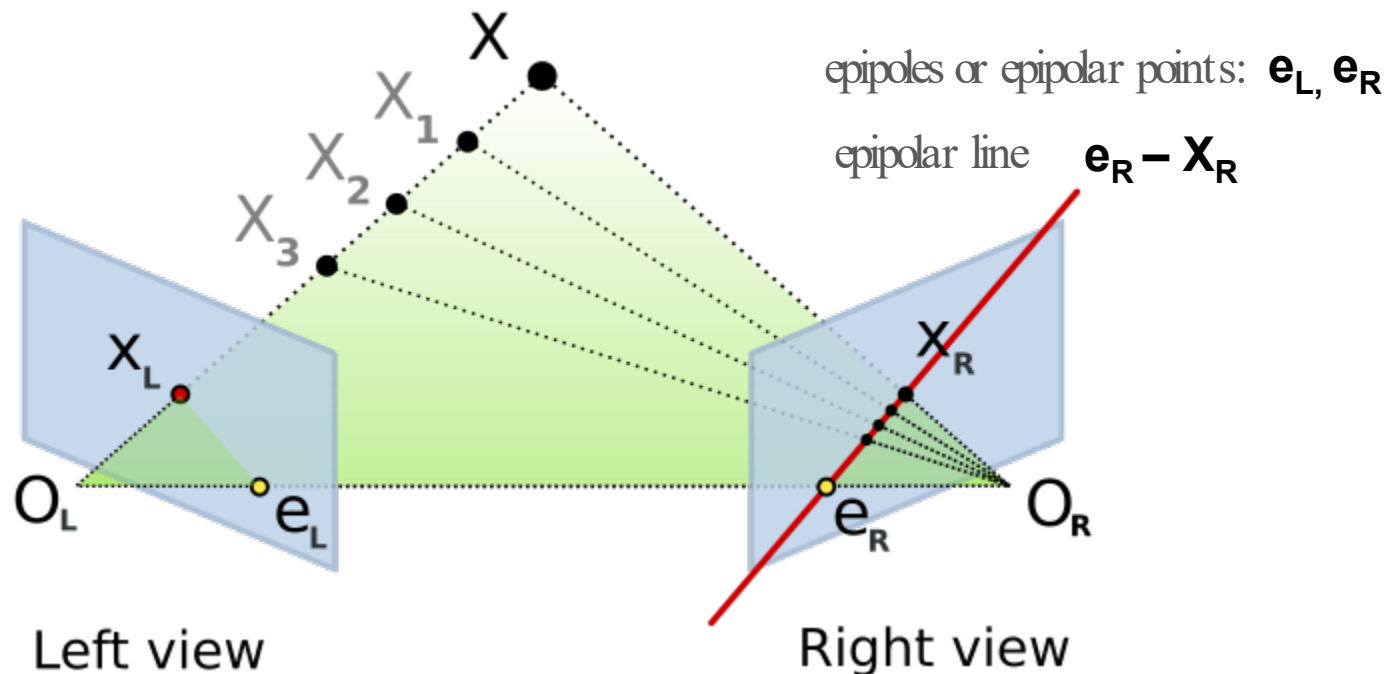
- **Epipolar geometry** is the geometry of stereo vision.
- When two cameras view a 3D scene from two distinct positions, there are a number of geometric relations between the 3D points and their projections onto the 2D images that lead to constraints between the image points.



Source: https://en.wikipedia.org/wiki/Epipolar_geometry

Stereo Vision. Epipolar Geometry

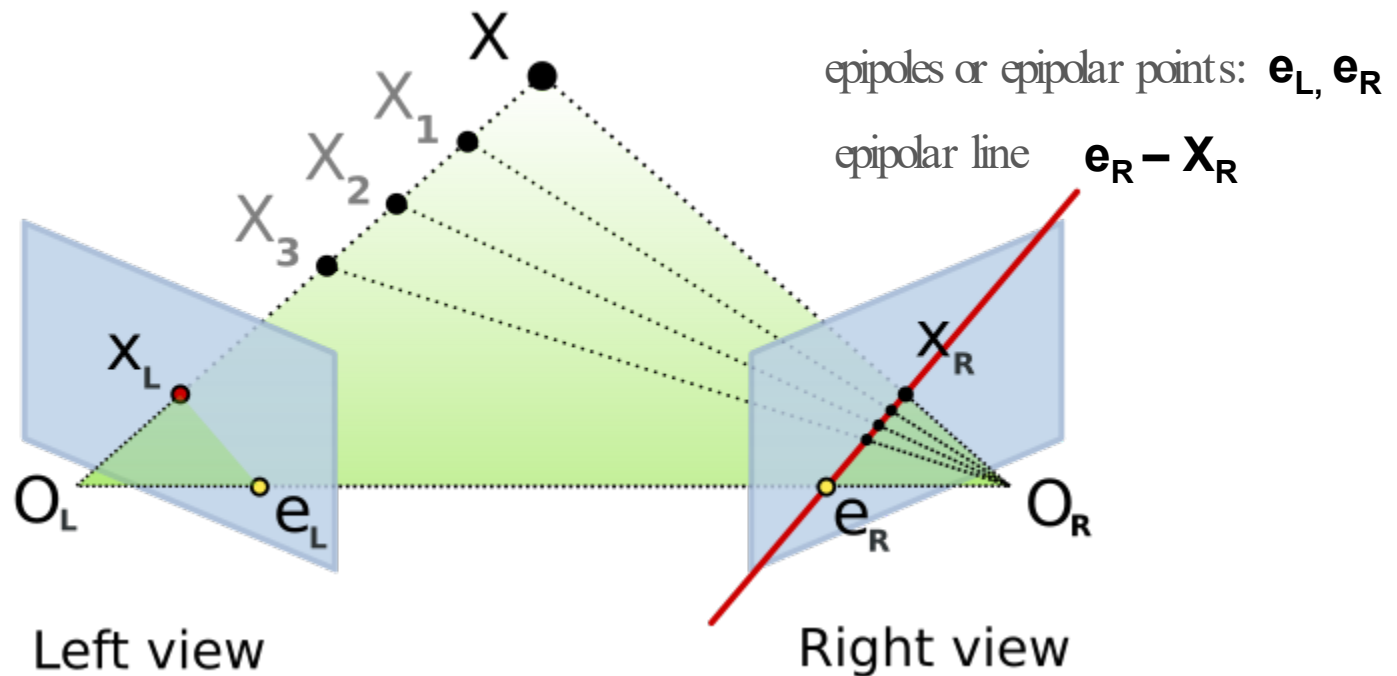
- **Epipolar geometry** is the geometry of stereo vision.
- **Epipolar constraint:** the projection of $\mathbf{X}$ on the right camera plane $\mathbf{X}_R$ must be contained in the $\mathbf{e}_R - \mathbf{X}_R$ epipolar line. All points $\mathbf{X}$, e.g. $\mathbf{X}_1, \mathbf{X}_2, \mathbf{X}_3$ on the $\mathbf{O}_L - \mathbf{X}_L$ line will verify that constraint.



Source: https://en.wikipedia.org/wiki/Epipolar_geometry

Stereo Vision. Epipolar Geometry

- **Epipolar geometry** is the geometry of stereo vision.
- **Epipolar constraints** can also be described by the **fundamental matrix**, or in the case of normalized image coordinates, the **essential matrix** between the two cameras.

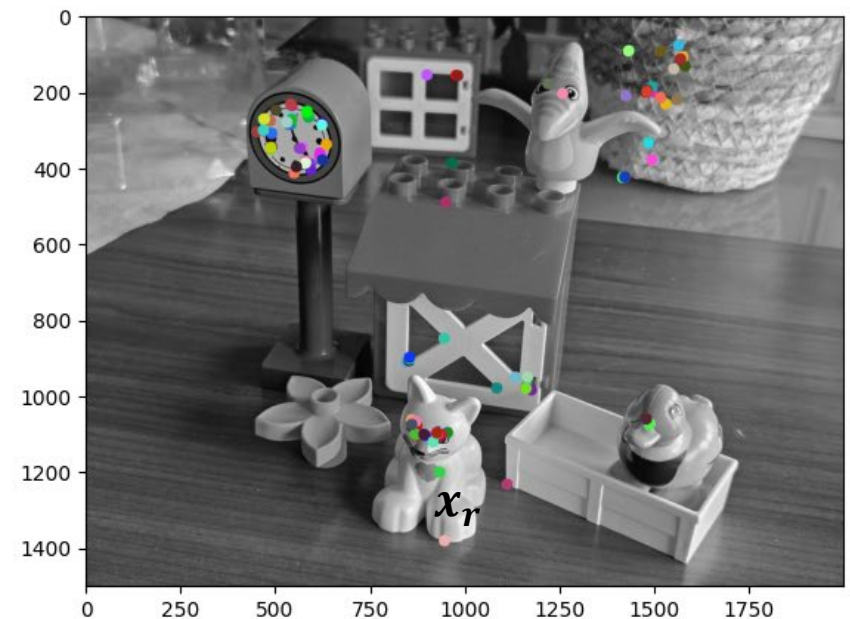
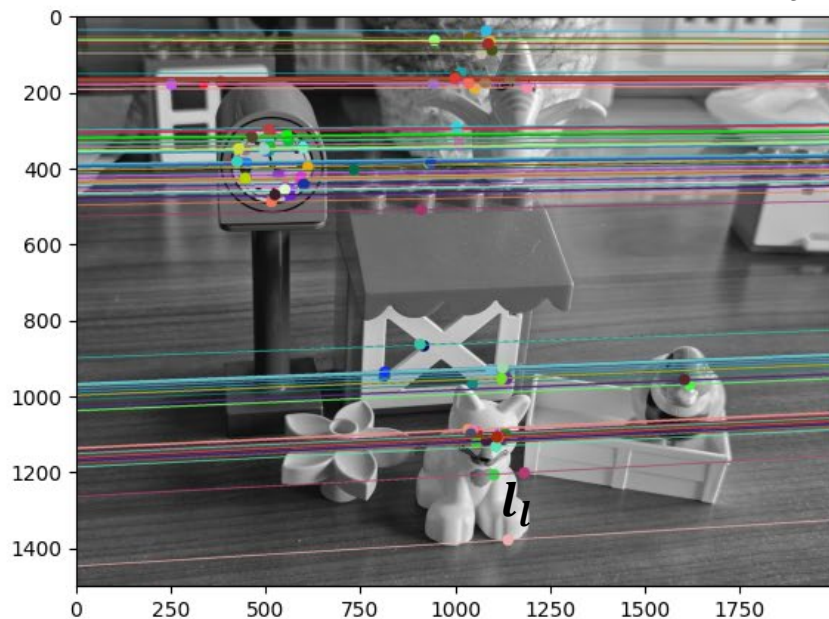


Source: https://en.wikipedia.org/wiki/Epipolar_geometry

Epipolar Geometry: The Essential Matrix

- The Essential Matrix ($\mathbf{E}$) is a matrix that encodes epipolar geometry. Given a point in one image, multiplying by the essential matrix will tell us the epipolar line in the second paired image:

$$l_l = \mathbf{E}x_r$$



Epipolar Geometry: The Essential Matrix

- The Essential Matrix ($\mathbf{E}$) is a matrix that encodes epipolar geometry. Given a point in one image, multiplying by the essential matrix will tell us the epipolar line in the second paired image:

$$\mathbf{E} = [t]_{\times} \mathbf{R} = \begin{bmatrix} 0 & -t_x & t_y \\ t_x & 0 & -t_z \\ -t_y & t_x & 0 \end{bmatrix}$$

- Problem: This matrix works in normalized camera coordinates, so the cameras need to be calibrated in order to transform points into pixel coordinates:

Need to transform from x_l pixel coordinates to x'_l normalized coordinates

$$x'_l = \mathbf{K}^{-1} x_l, \quad x'_r = \mathbf{K}^{-1} x_r \\ (x'_r)^T \mathbf{E} x'_l = 0$$

Epipolar Geometry: The Fundamental Matrix

- The Fundamental Matrix ($\mathbf{F}$) is similar to the essential matrix, as it also encodes epipolar geometry, but it works in the image plane (in pixel units).
- Given x_l, x_r corresponding points in a stereo image pair, $\mathbf{F} x_r$ represents the epipolar line on which x_l must lie. That means:

$$x_l^T \mathbf{F} x_r = 0$$

- Assuming both images share identical intrinsic parameters ($\mathbf{K}$). We can obtain the fundamental matrix from the essential one, since:

$$\mathbf{E} = \mathbf{K}^T \mathbf{F} \mathbf{K}$$

it is possible to write

$$\mathbf{F} = \mathbf{K}^{-T} \mathbf{E} \mathbf{K}^{-1} = \mathbf{K}^{-T} [\mathbf{t}]_{\times} \mathbf{R} \mathbf{K}^{-1}$$

Camera Intrinsics and Rectification

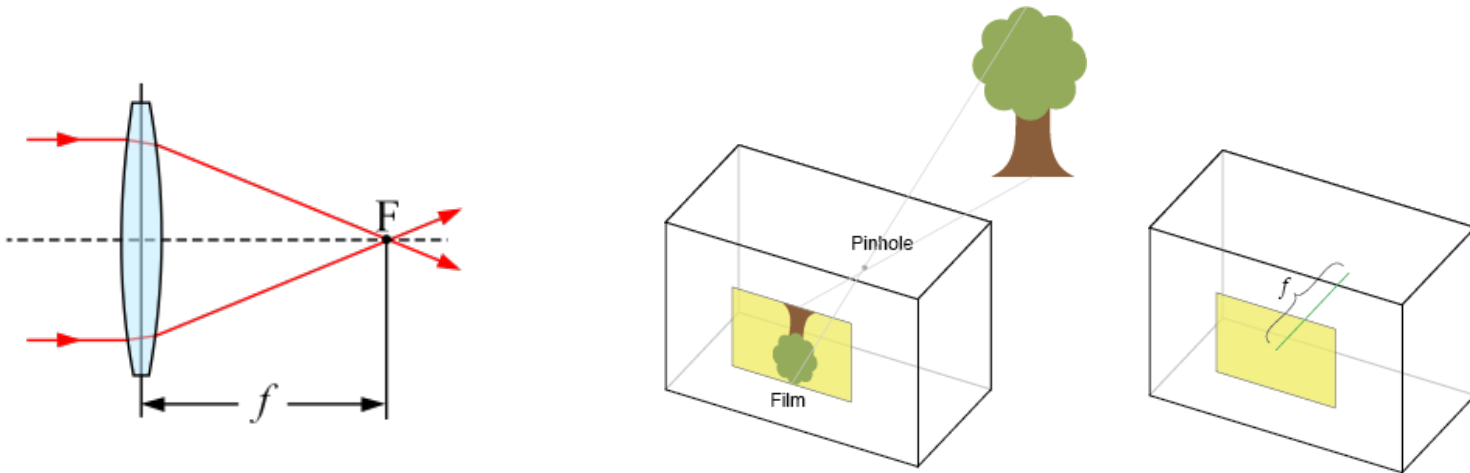
- The intrinsic matrix can be parameterized as:

$$\mathbf{K} = \begin{pmatrix} f_x & s & x_0 \\ 0 & f_y & y_0 \\ 0 & 0 & 1 \end{pmatrix}$$

- Each intrinsic parameter describes a geometric property:
 - **Focal length** f_x and f_y
 - **Principal point offset**, x_0, y_0
 - **Axis Skew**, s
- Let's briefly define them ...

Camera Intrinsics. Focal Length

- Focal length f_x and f_y
- It is the distance between the pinhole and the film (a.k.a. *image plane*).
 - In a true pinhole camera, both have the same value.

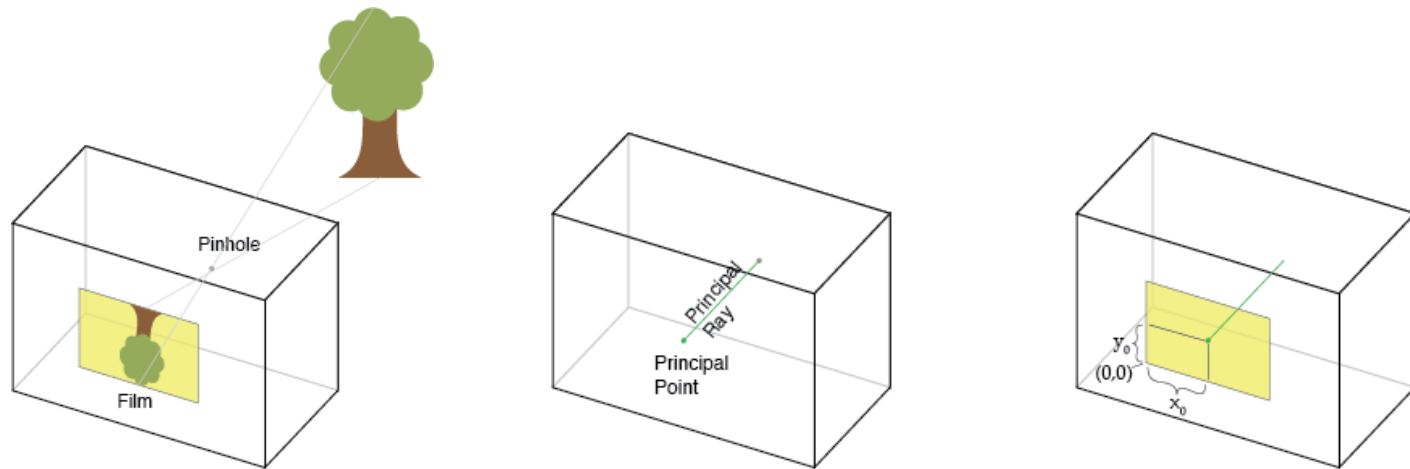


- It allows to convert pixel units to world units (e.g. mm):

$$\mathbf{F}_x = f_{lens} \frac{W \text{ (image width in px)}}{w \text{ (sensor width in mm)}}, \quad \mathbf{F}_y = f_{lens} \frac{H}{h}, \quad \mathbf{X}_0 = x_0 \frac{W}{w}, \quad \mathbf{Y}_0 = y_0 \frac{H}{h}$$

Camera Intrinsics. Principal Point Offset & Axis Skew

- **Principal point offset** (x_0, y_0)
- The camera's "*principal axis*" is the line perpendicular to the image plane that passes through the pinhole.

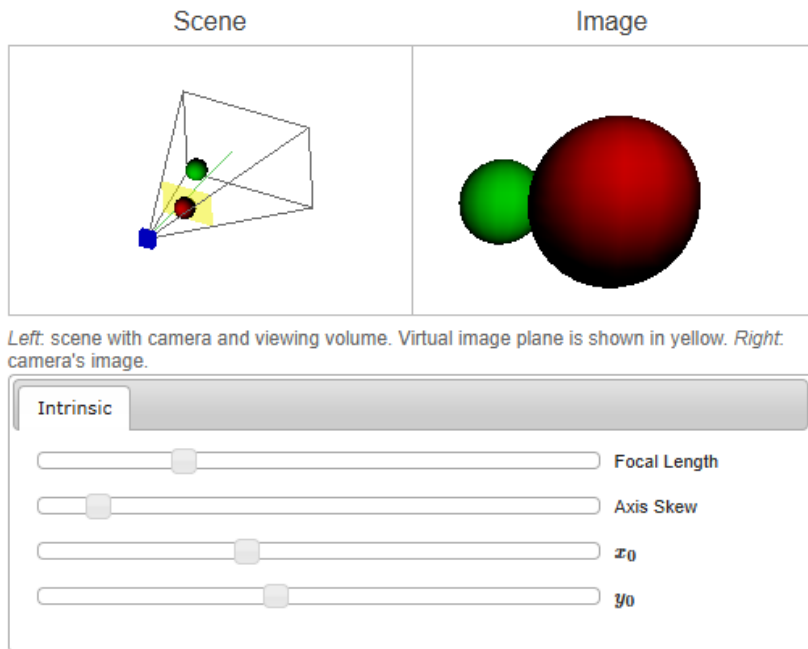


- **Axis Skew, s**
 - Axis skew causes shear distortion in the projected image
- You can play with camera intrinsic parameters!
 - <https://ksimek.github.io/2013/08/13/intrinsic/>

Source: <https://ksimek.github.io/2013/08/13/intrinsic/>

Camera Intrinsics

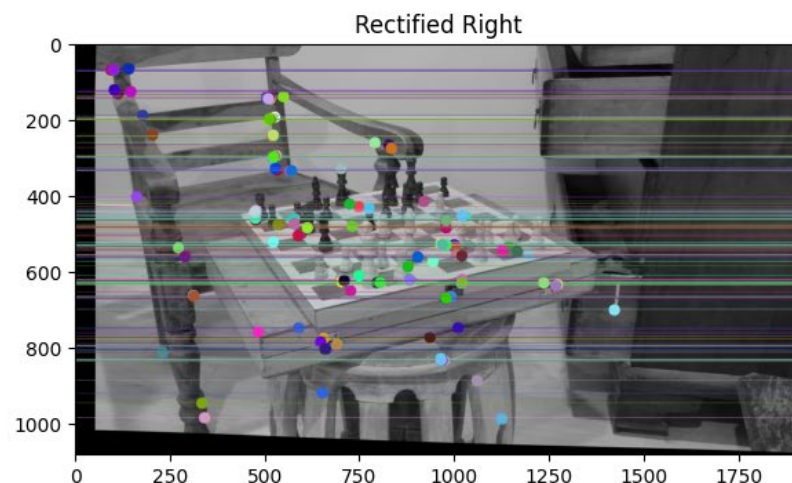
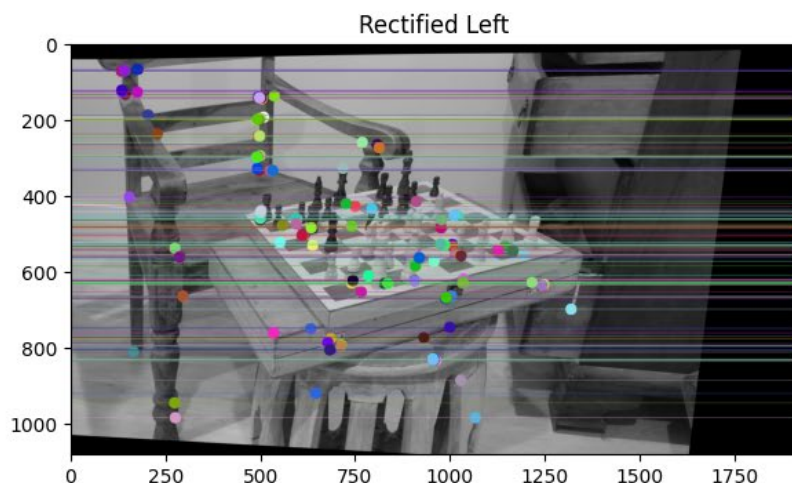
- Let's play a bit with camera intrinsics parameters!



<https://ksimek.github.io/2013/08/13/intrinsic/>

Stereo Vision. Rectification

- Rectification is a geometric transformation (homography) applied to stereo images so that **epipolar lines become aligned and horizontal**.
- After rectification:
 - Any point in the left image corresponds to a point **on the same row (same y-coordinate)** in the right image.
 - So correspondence search reduces to **1D search along a row**, instead of 2D search along an arbitrary epipolar line.



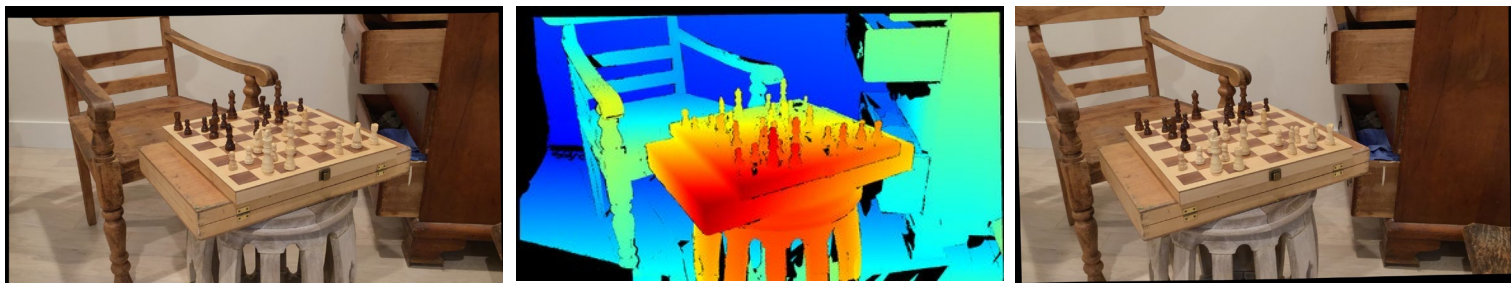
Source: <https://vision.middlebury.edu/stereo/data/scenes2021/data/chess1>

Stereo Vision. Disparity

- The **disparity** is the apparent motion of objects between a pair of stereo images. It can be expressed simply as:

$$d = x_l - x_r$$

- Given a pair of stereo images, to compute the **disparity map**, we first match every pixel in the left image with its corresponding pixel in the right image. Then we compute the distance for each pair of matching pixels. Finally, the disparity map is obtained by representing such distance values as an intensity image.



Source: <https://vision.middlebury.edu/stereo/data/scenes2021/data/chess1>

Stereo Vision. Disparity



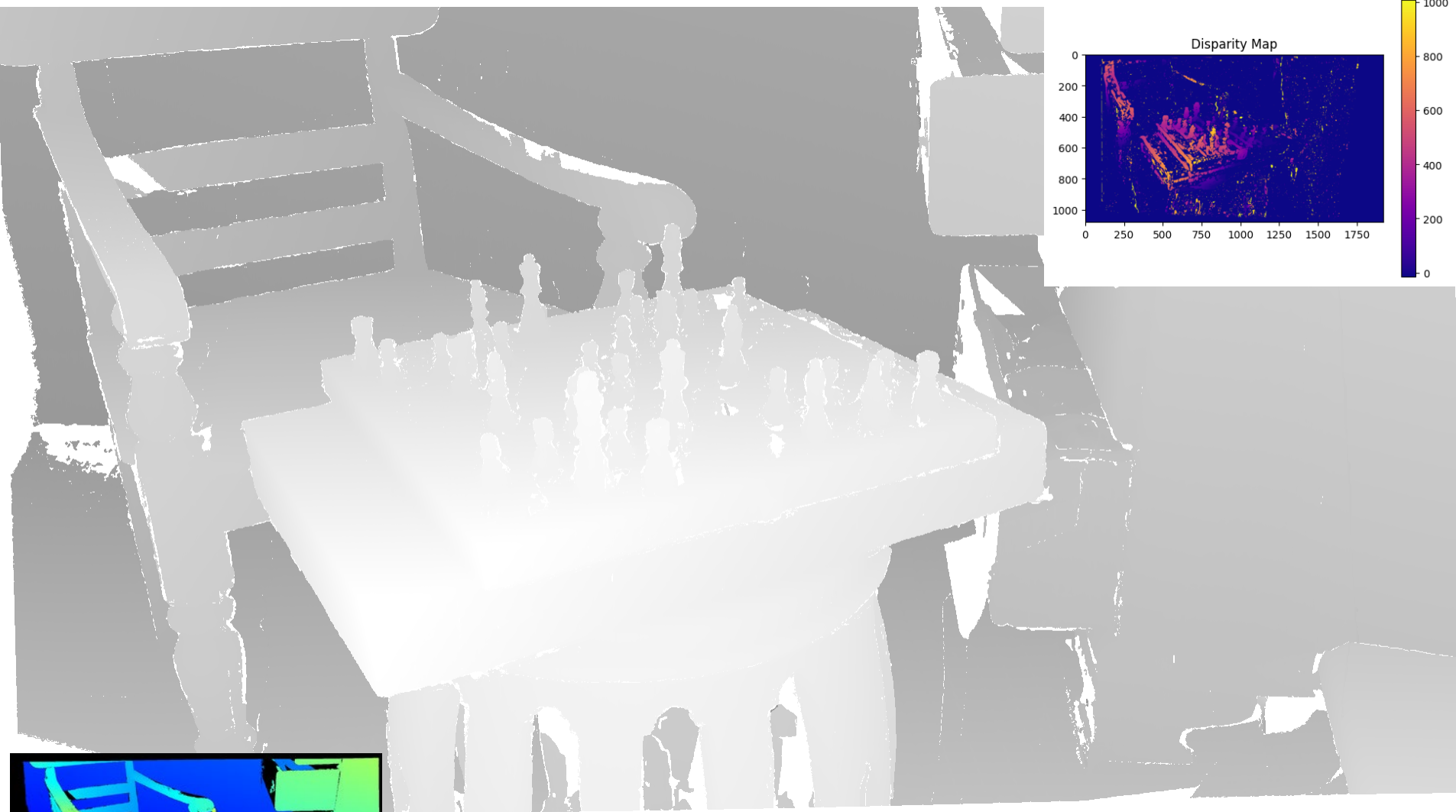
Source: <https://vision.middlebury.edu/stereo/data/scenes2021/data/chess1>

Stereo Vision. Disparity



Source: <https://vision.middlebury.edu/stereo/data/scenes2021/data/chess1>

Stereo Vision. Disparity



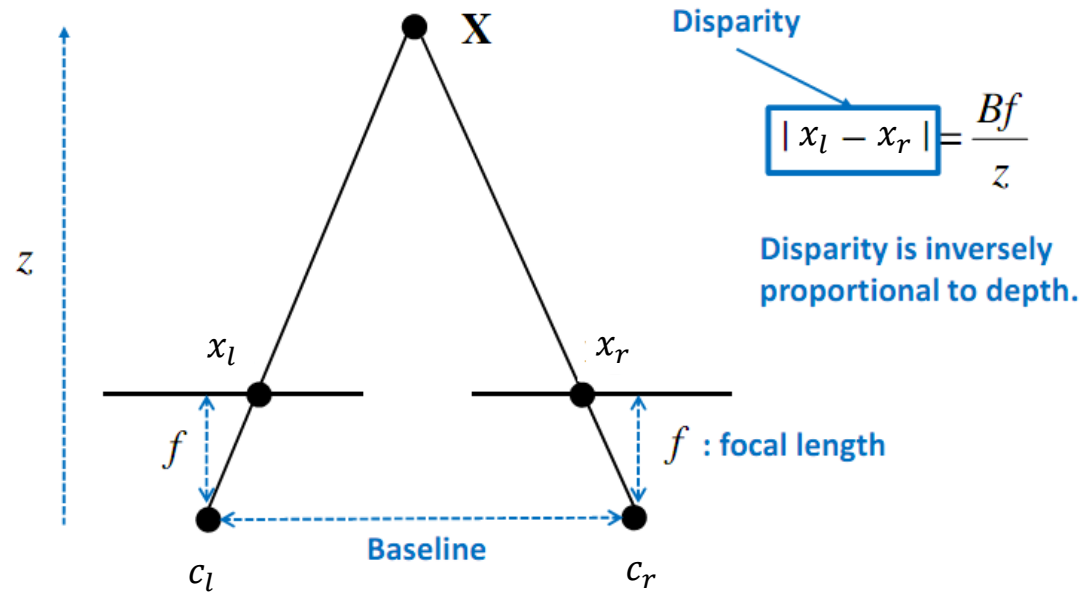
Source: <https://vision.middlebury.edu/stereo/data/scenes2021/data/chess1>

Stereo Vision. Depth (Z) from disparity

$$d = x_l - x_r$$

(through triangle similarity..)

$$x_l = \frac{fX}{Z}, \quad x_r = \frac{f(X-B)}{Z}$$
$$d = \frac{fX}{Z} - \frac{f(X-B)}{Z} = \frac{fB}{Z} \Rightarrow Z = \frac{fB}{d}$$



Source: Adapted from Aissaoui et al. Reconstruction 3D de visages dans un système de stéréovision basée sur les propriétés du visage

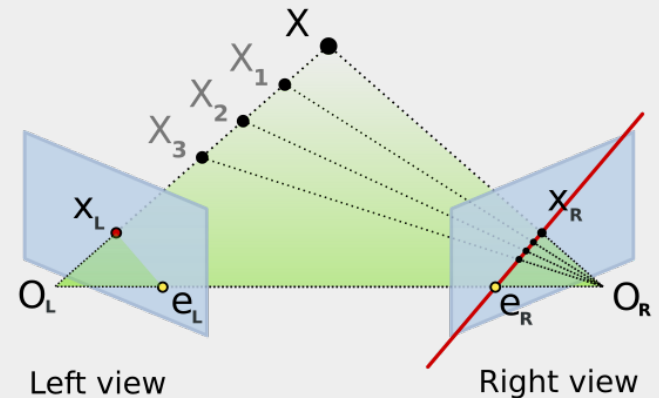
Green AI in Stereo Feature Matching (Epipolar Geometry)

- 🌱 Geometry reduces search space: Instead of scanning the whole image for matches, epipolar lines constrain the search to 1D curves.
 - Less computation = less wasted energy.
- 🌿 Rectification simplifies further → If images are rectified, matches are only searched along horizontal lines. This makes stereo matching massively more efficient, lowering runtime and idle power.
- 🌱 RANSAC with epipolar constraint = fewer false matches → By rejecting outliers early, we avoid propagating bad correspondences into expensive later steps. Saves energy on reconstruction and improves photogrammetry accuracy.
- 🌿 Smarter keypoint sampling → Using geometry, we don't need to keep all keypoints (which are costly to describe). Selecting only geometrically consistent keypoints reduces descriptor computation overhead.
- 🌱 Parallelization-friendly → Epipolar-constrained matching distributes well across GPU cores because each epipolar line can be processed independently.
 - Less idle time = less energy wasted.

Epipolar Geometry Exercise

stereo_reconstruction.ipynb

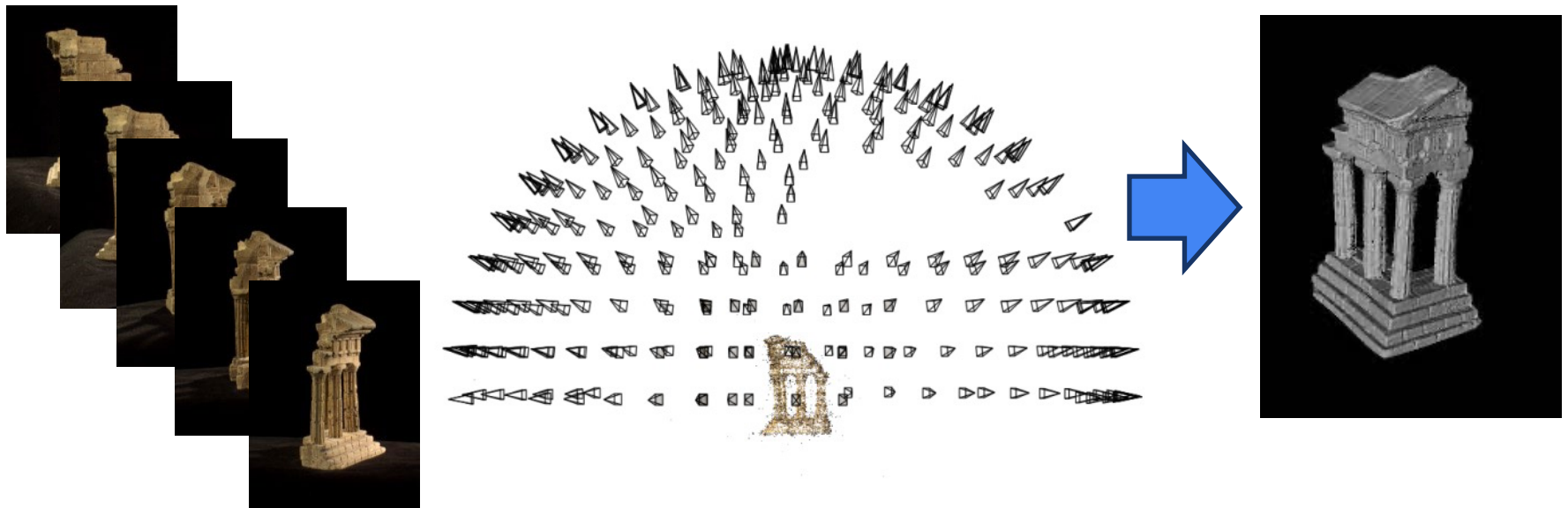
- Let's find some correspondences between images ...



Multi-View Stereo (MVS)

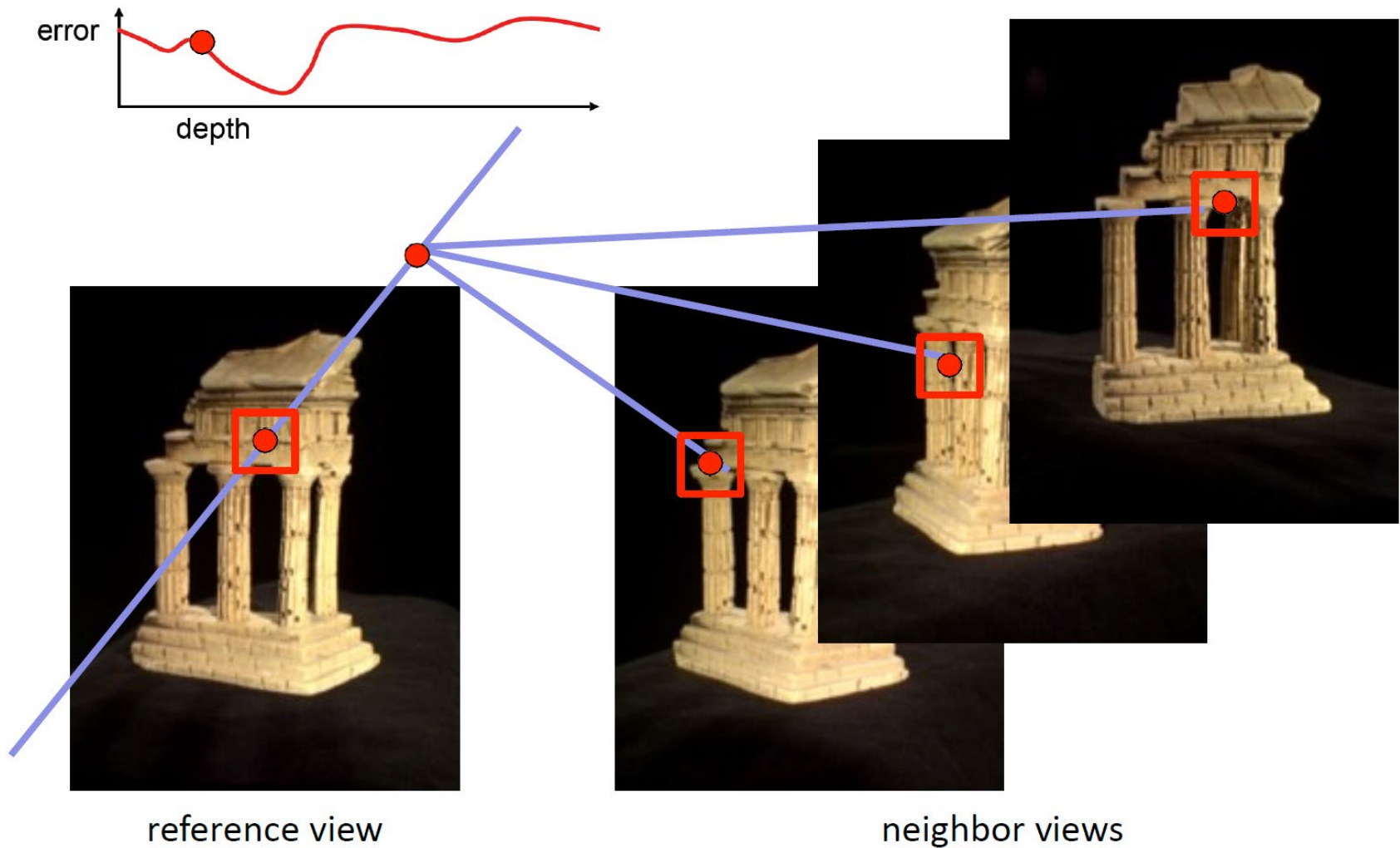
Multi-View Stereo (MVS)

- **Problem formulation:** given several images of the same object or scene, compute a representation of its 3D shape.



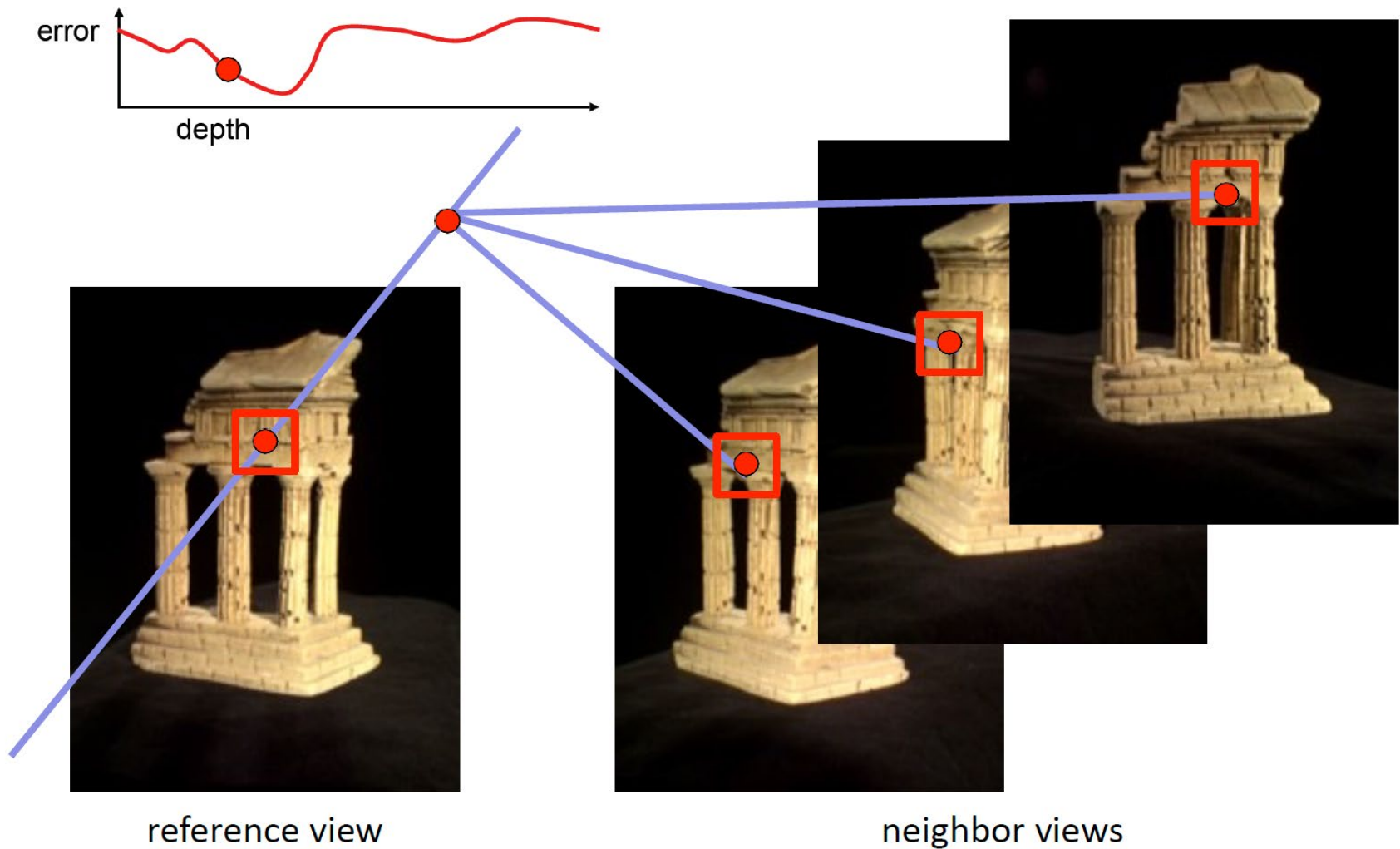
Source: Roni Sengupta Lecture 20: Multi-View Stereo (MVS)

Multi-View Stereo (MVS)



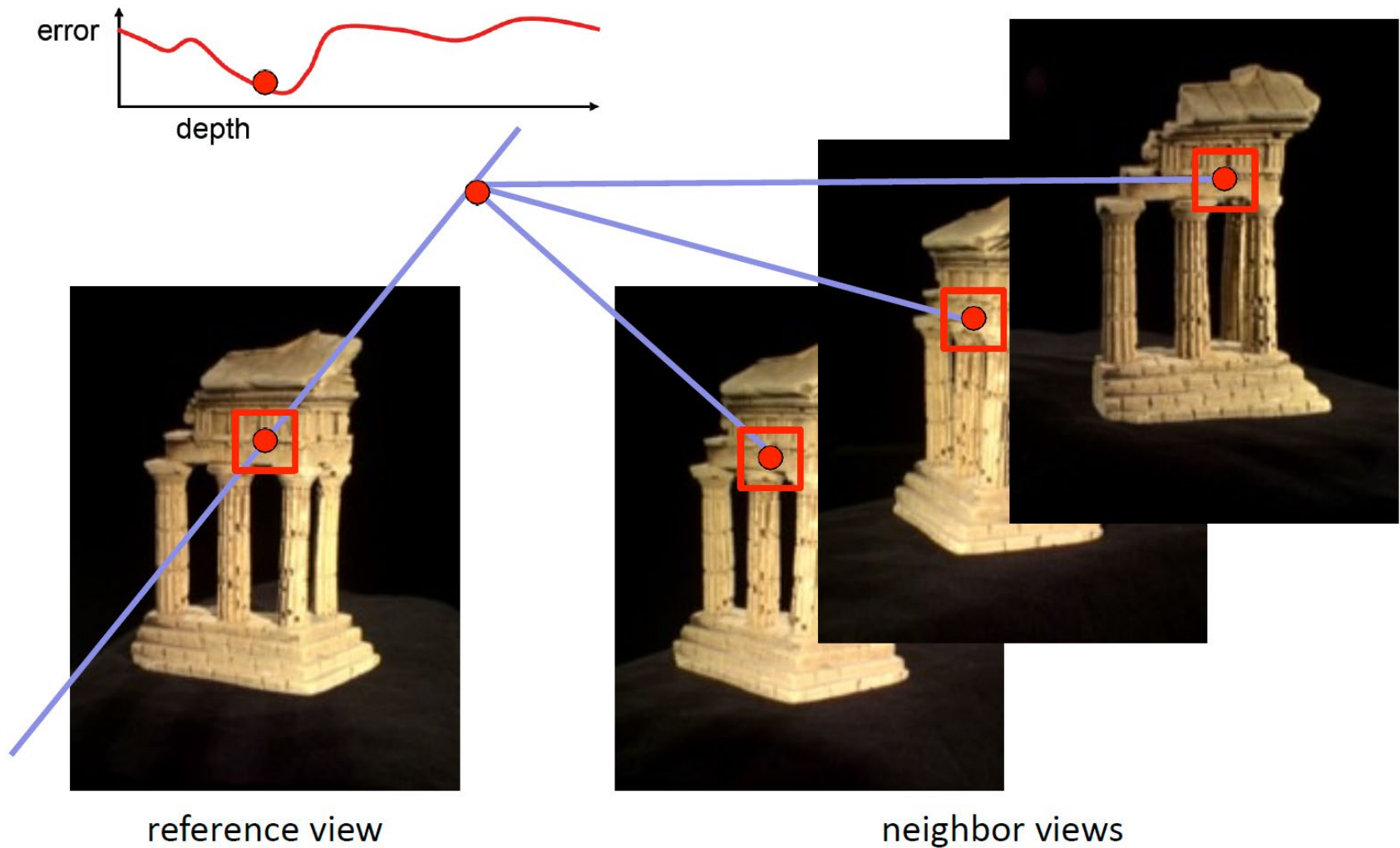
Source: Y. Furukawa

Multi-View Stereo (MVS)



Source: Y. Furukawa

Multi-View Stereo (MVS)



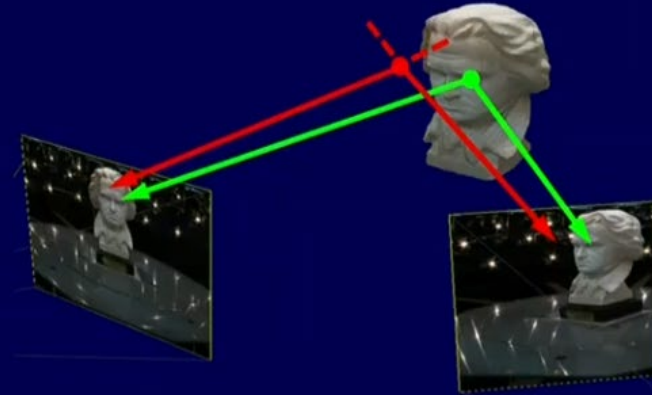
Source: Y. Furukawa

Multi-View Stereo (MVS). Photo-consistency

- **Photo-consistency** related to a 3D point in the real world refers to **how consistent its color or intensity** is when observed **from multiple camera views**.

Photoconsistency function:

$$\rho : \mathbb{R}^3 \rightarrow [0, 1]$$



Determine a surface S of optimal photoconsistency by minimizing

$$E(S) = \int_S \rho \, dA$$



Source: Multiple View Geometry (3D Computer Vision). Prof. Dr. Daniel Cremers (TU München)

Multi-View Stereo (MVS)

- Stereo = 2 views → depth with limitations (gaps, occlusions)
 - Adding more views improves coverage, robustness, and accuracy.
- Adding more views ...
 - ... estimate correspondences across many images.
 - ... each correspondence triangulates to a 3D point.
 - ... more views = better accuracy + aid for disambiguation of occlusions.
- The **challenge** remains to find correspondences across many images while maintaining global consistency.
 - **Epipolar geometry generalizes**: every pair of images still obeys the epipolar constraint.

Multi-View Stereo (MVS). Multibaseline Stereo

- The idea is to minimize the cost (SSD/SAD/NCC) over all views, not just 2!
 - SSD (Sum of Squared Differences)
 - SAD (Sum of Absolute Differences)
 - ZNCC (Zero-mean Normalized Cross Correlation)

$$\text{SSD}(I_1, I_2) = \sum (I_1 - I_2)^2$$

$$\text{ZNCC}(I_1, I_2) = \frac{\sum ((I_1 - \mu_1)(I_2 - \mu_2))}{\sigma_1 \sigma_2}$$

structure similarity ↗

$$C(p, d) = \sum_{k=1}^N \sum_{(u,v) \in W} \left(I_{\text{ref}}(x + u, y + v) - I_k(\pi_k(d, x + u, y + v)) \right)^2$$

p : the pixel in the **reference image** where we want to estimate depth/disparity.

d : candidate disparity (or depth hypothesis).

W : a support window around p .

I_{ref} : intensity values in the reference image.

$\pi_k(d, x + u, y + v)$: reprojection of the 3D point corresponding to pixel $(x + u, y + v)$ with disparity d , into camera k .

Multi-View Stereo (MVS). Multibaseline Stereo - Limitations

- **Limitations:**
 - The result looks more like a depth map (rather than an “object model”).
 - Do not work for widely distributed views.
- **Visibility (problem):** Which points are visible in which images?

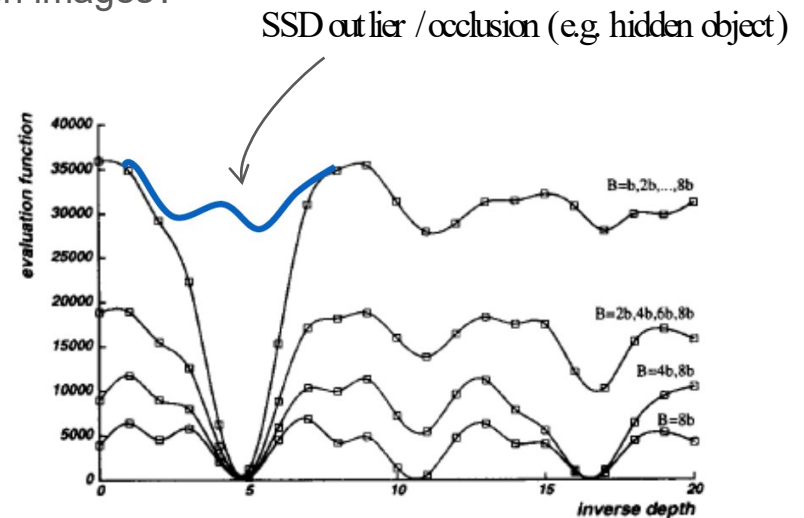
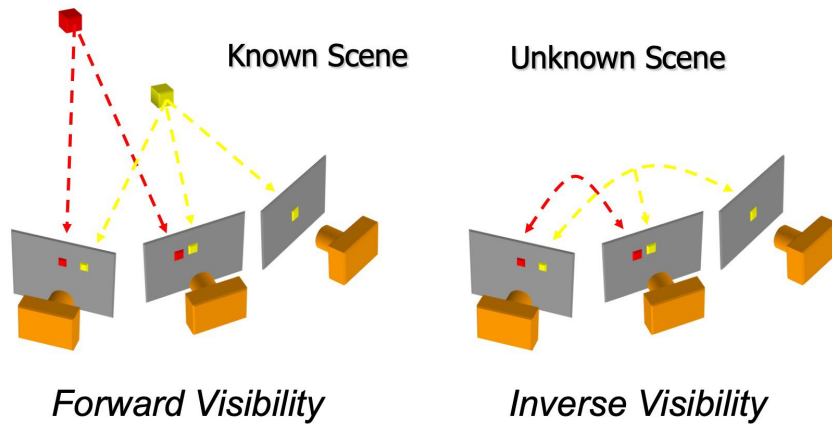


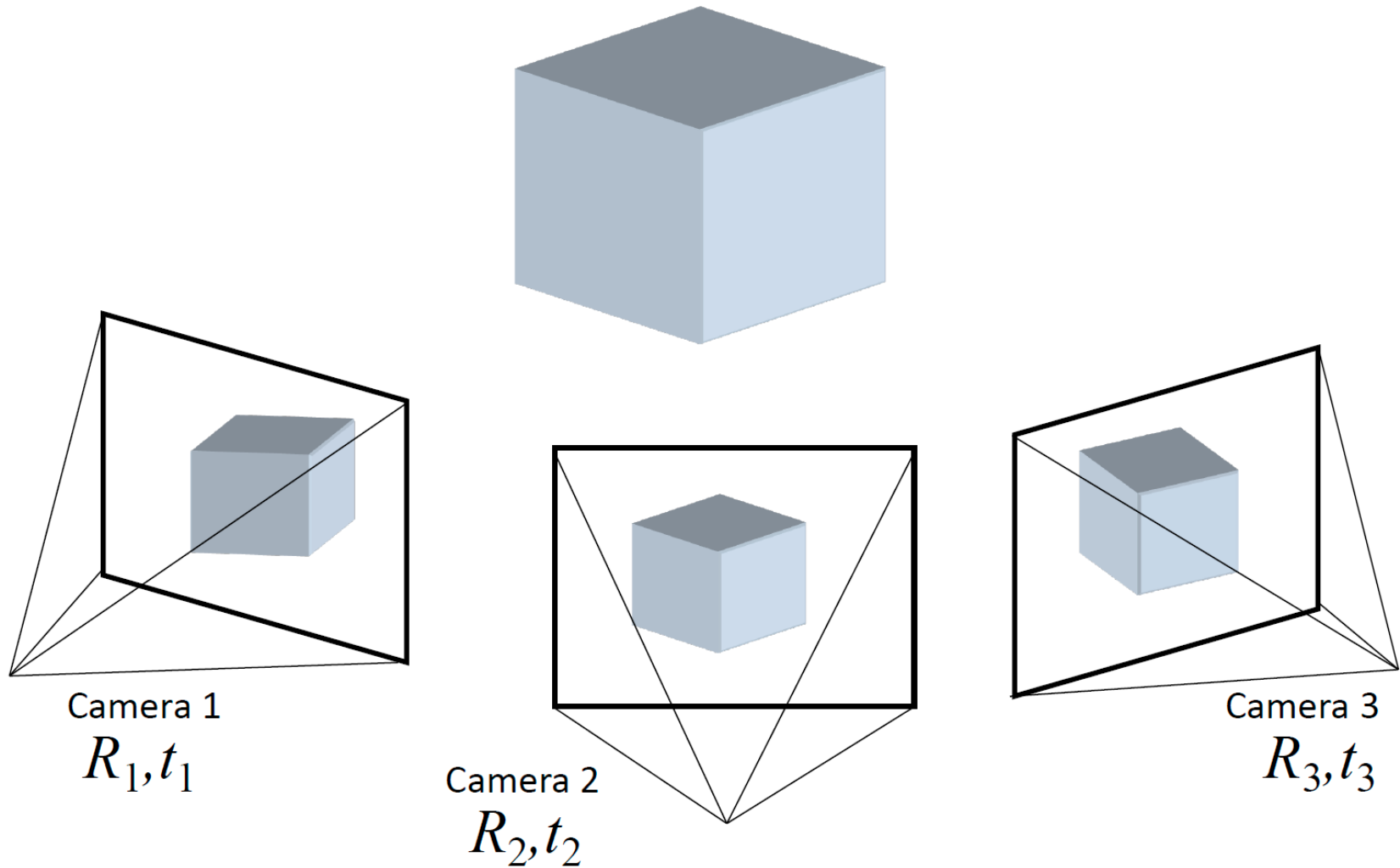
Fig. 7. Combining multiple baseline stereo pairs.

Source: Roni Sengupta Lecture 20: Multi-View Stereo (MVS)

Source: M. Okutomi and T. Kanade, *A Multiple-Baseline Stereo System*, IEEE Trans. On Pattern Analysis and Machine Intelligence, 15(4):353-363 (1993).

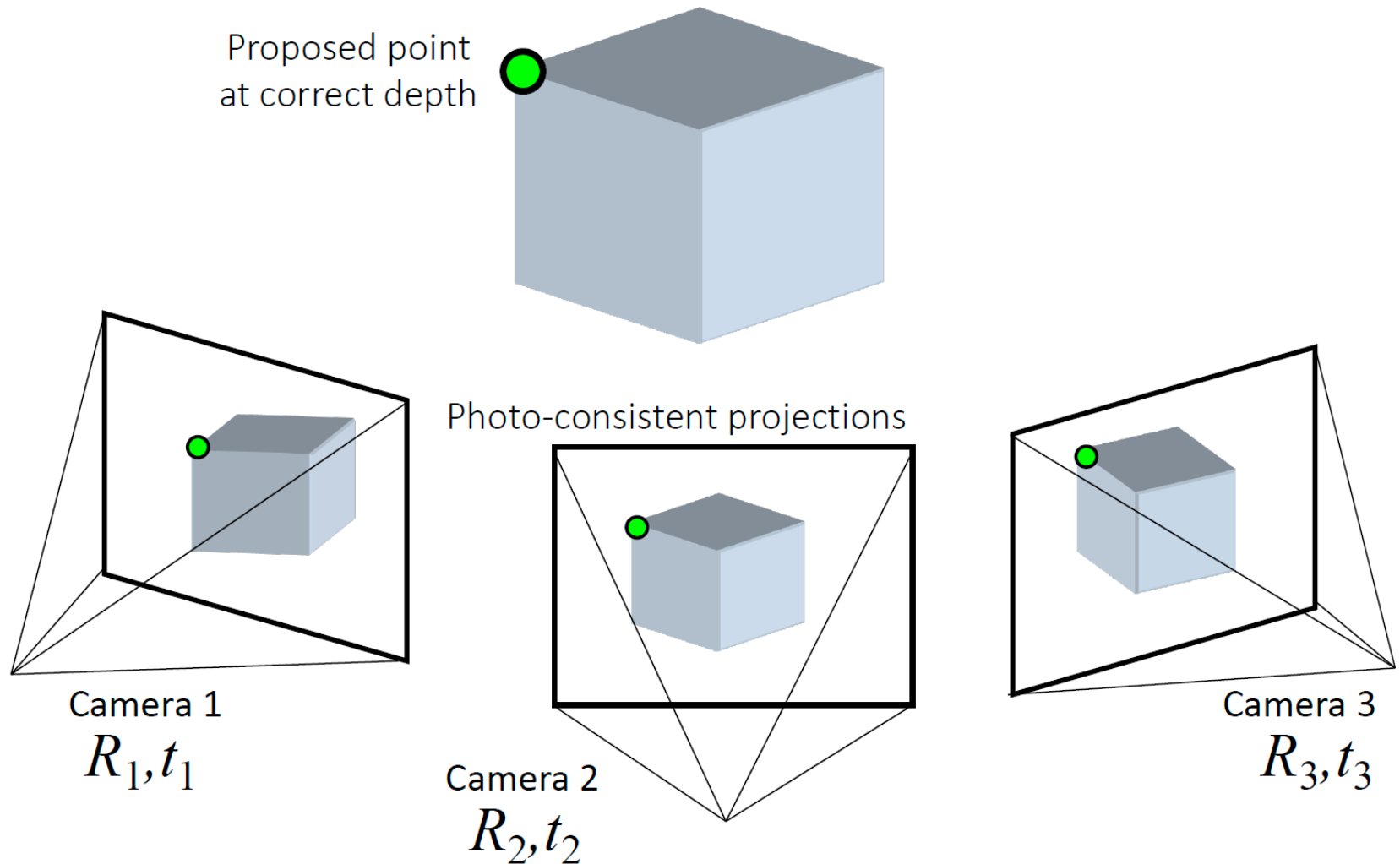
- ➔ Use NCC instead of SSD and ignore NCC values $> \text{threshold}$ [Hernandez & Schmitt 03]
- SSD is very sensitive to outliers

MVS Advanced Approaches. Plane Sweep Stereo



Source: Roni Sengupta Lecture 20: Multi-View Stereo (MVS)

MVS Advanced Approaches. Plane Sweep Stereo



Source: Roni Sengupta Lecture 20: Multi-View Stereo (MVS)

MVS Advanced Approaches. Plane Sweep Stereo

Proposed point at
incorrect depth

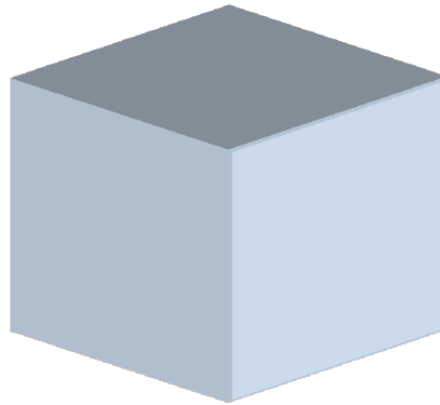
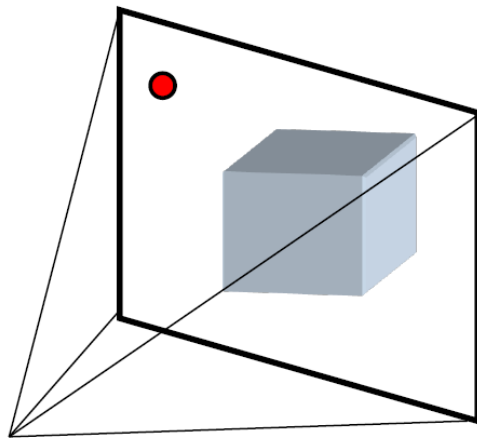
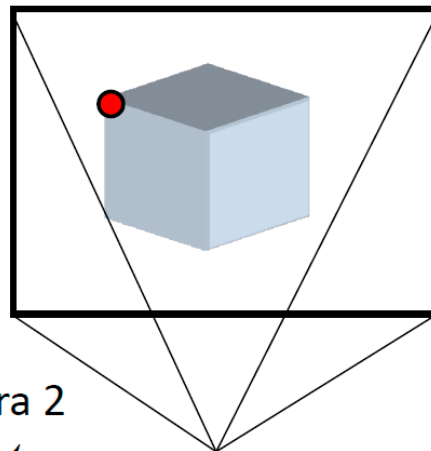


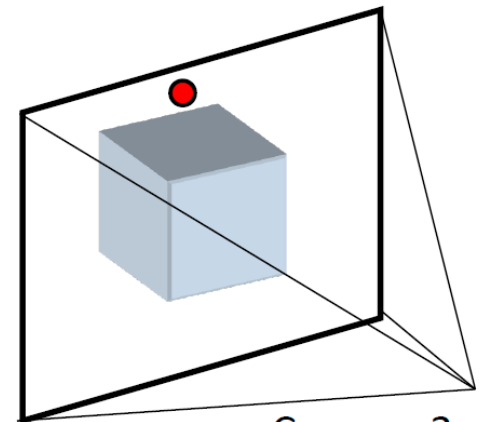
Photo-inconsistent projections



Camera 1
 R_1, t_1



Camera 2
 R_2, t_2

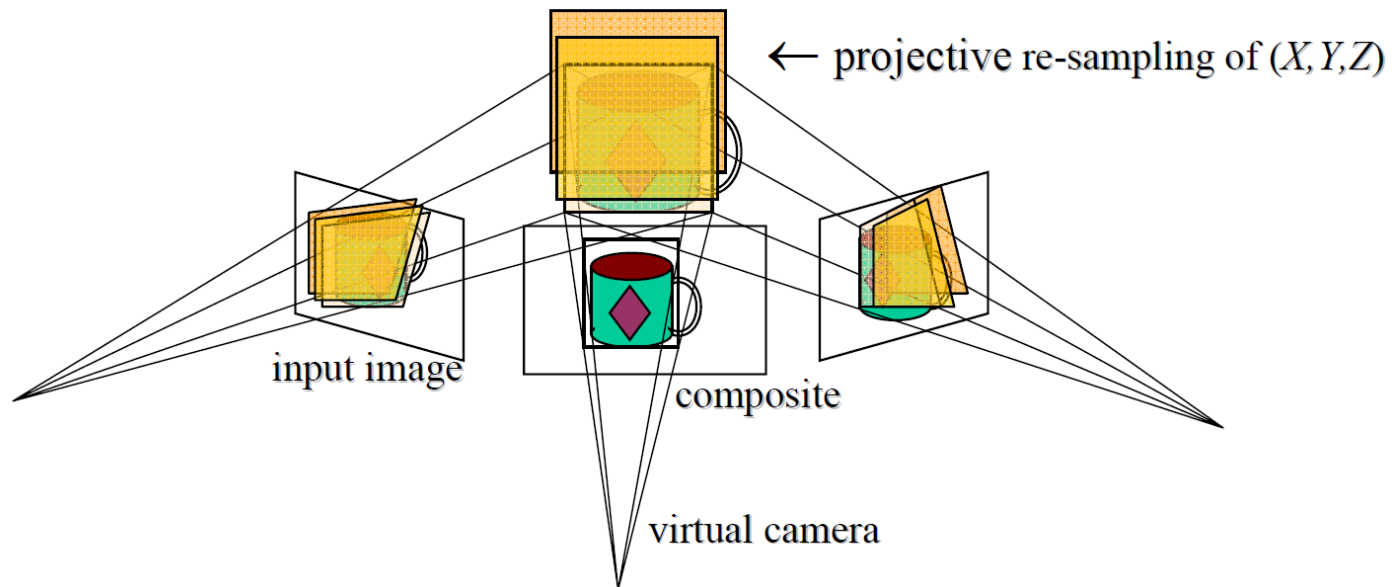


Camera 3
 R_3, t_3

Source: Roni Sengupta Lecture 20: Multi-View Stereo (MVS)

MVS Advanced Approaches. Plane Sweep Stereo

- Sweep family of planes parallel to the reference camera image plane.
- Reproject neighbors onto each plane and compare reprojections.
- At each plane, compare reference image pixel values with warped images.
 - If pixel intensities match well, the hypothesized depth is likely correct for that point.



Source: Roni Sengupta Lecture 20: Multi-View Stereo (MVS)

MVS Advanced Approaches. Plane Sweep Stereo (Example)

- For a particular **depth sweep**, some regions in the average image appears sharp, i.e. **photo-consistent**.



Left neighbor



Reference image



Right neighbor



Planar image reprojections swept over depth
(averaged)

MVS Advanced Approaches. Plane Sweep Stereo



Left neighbor



Reference image



Right neighbor



Left neighbor projected into
reference image



Average images on each plane

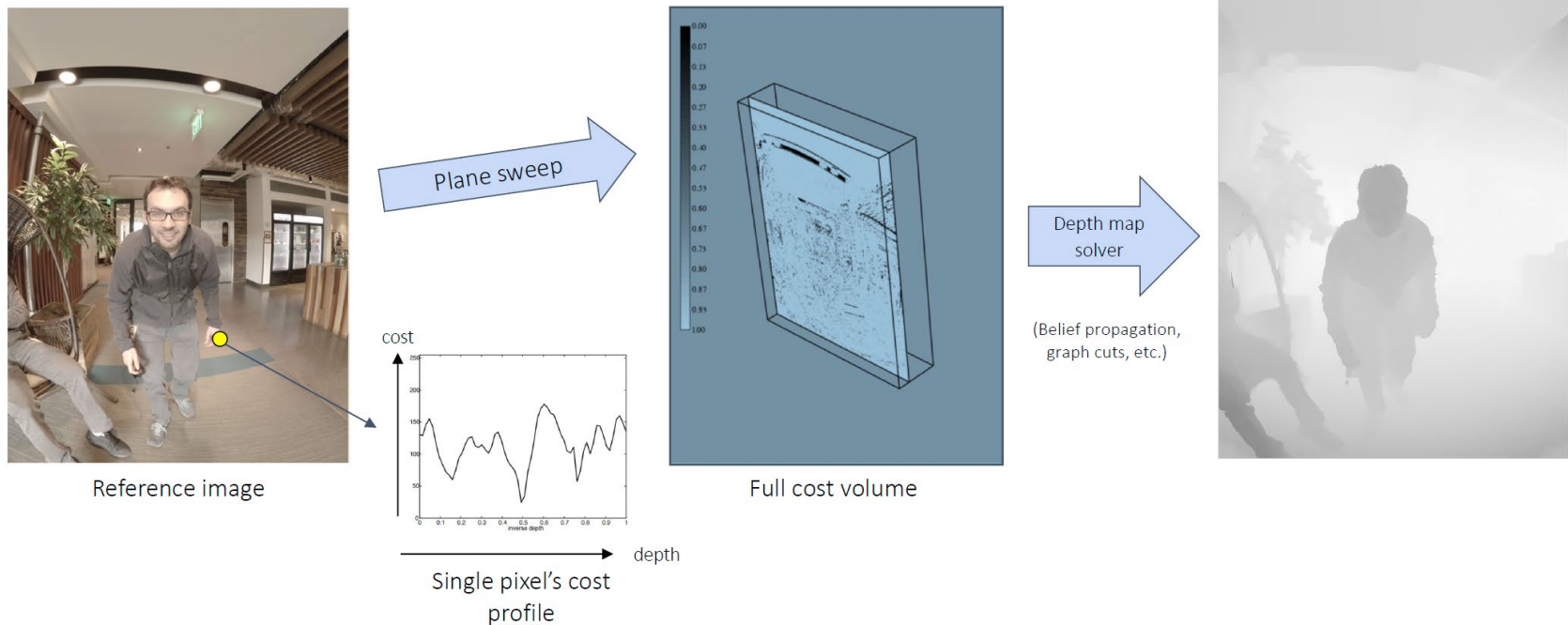


Right neighbor projected into
reference image

Source: Roni Sengupta Lecture 20: Multi-View Stereo (MVS)

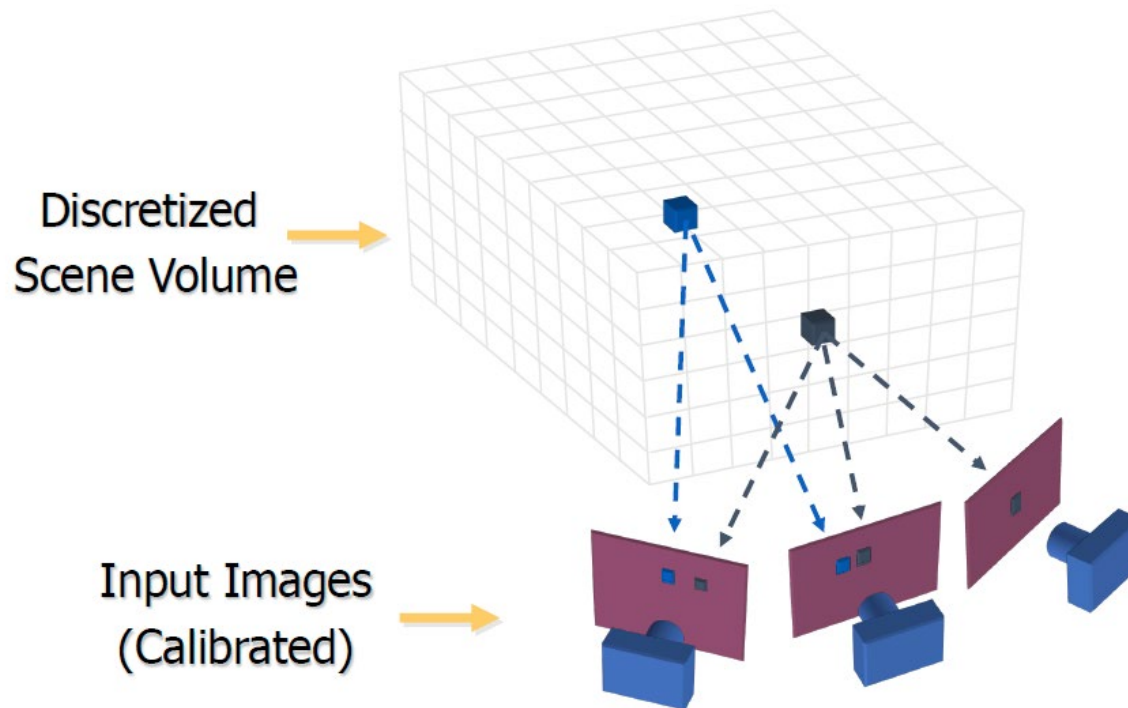
MVS Advanced Approaches. Plane Sweep Stereo

- Plane sweeping produces a full cost volume.
- **Cost Volumes** → **Depth Map**



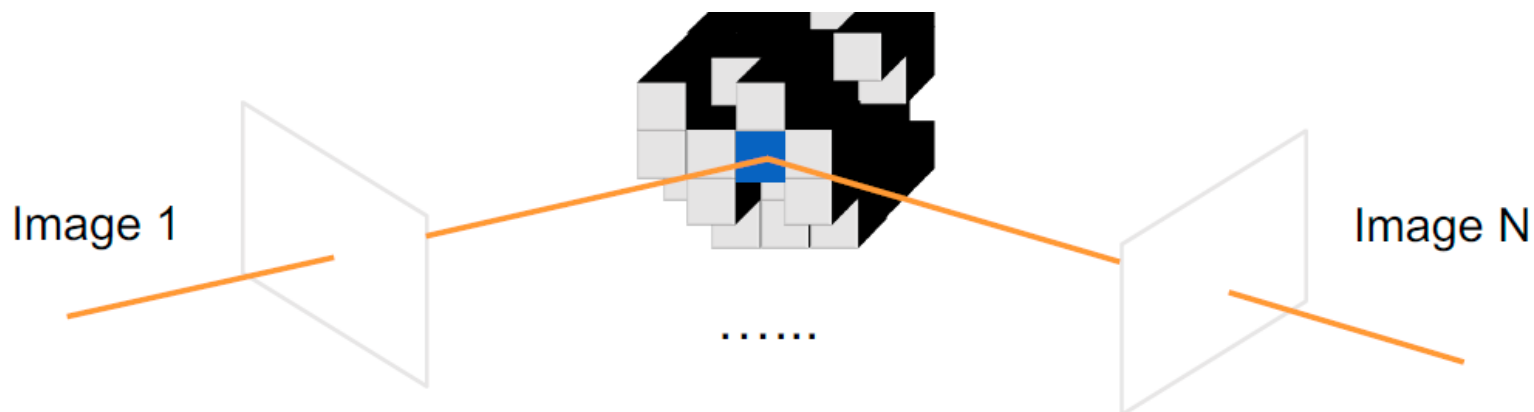
Multi-View Stereo (MVS). Volumetric Stereo

- **Key idea:** assign RGB values to photo-consistent voxels in V with images.



Source: Roni Sengupta Lecture 20: Multi-View Stereo (MVS)

Multi-View Stereo (MVS). Volumetric Stereo (Space Carving)



- Main idea of the space carving algorithm:
 - Initialize to a volume V containing the scene
 - Choose a voxel v on the outside of that volume:
 - Project v to visible input images
 - Carve if not photo-consistent
 - Repeat until convergence



Input Image (1 of 45)



Reconstruction

Source: K. N. Kutulakos and S. M. Seitz, A Theory of Shape by Space Carving, ICCV 1999

Multi-View Stereo (MVS). Patch-based MVS

- A patch-based MVS algorithm that is the core reconstruction step and reconstructs **a set of oriented points (patches, or surfels)** covering the surface of an object or scene.
- Conversion of patches into a mesh.
 - MVS algorithm that refines the mesh.

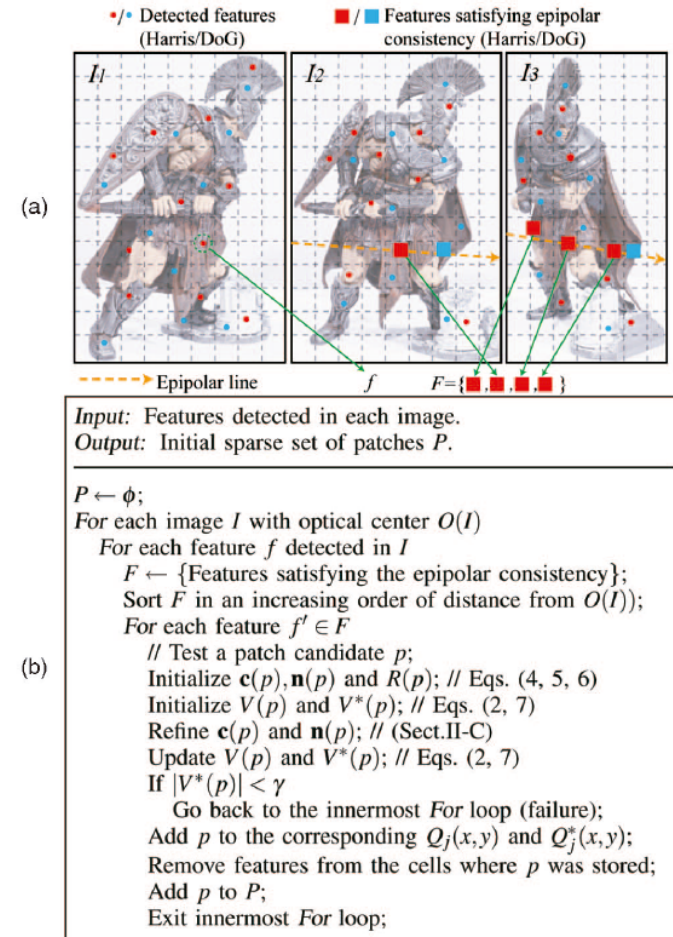
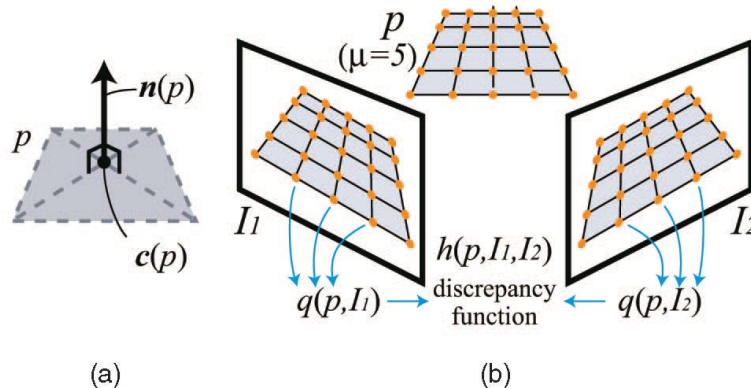


Fig. 4. Feature matching algorithm. (a) An example showing the features $f' \in F$ satisfying the epipolar constraint in images I_2 and I_3 as they are matched to feature f in image I_1 (this is an illustration only, not showing actual detected features). (b) The matching algorithm.

Multi-View Stereo (MVS). Summary of Classic Techniques

- Depth-map based methods
 - Estimate a per-view depth map using plane sweeping or patch matching via a 3D cost volume.
 - Leverage a 3D cost volume to estimate depth maps ...
 - ... then fuse all depth maps into a dense 3D point cloud or mesh.
 - *A Space-Sweep Approach to True Multi-Image Matching* [Collins1996]
- Voxel-based methods.
 - Divide space into a voxel grid, label voxels as free/occupied (e.g. space carving) → memory-intensive.
 - *Space Carving* [Kutulakos & Seitz, 2000]
 - *Photorealistic scene reconstruction by voxel coloring*. [Seitz & Dyer, 1999]
- (Surfels) Patch-based methods
 - Initialize patches in 3D and grow them by checking consistency across views.
 - Efficient & Scalable
 - Due to the affine matching, the expansion is limited to a grid of rectangular regions in image space.
 - PMVS - *Patch-based Multi-View Stereo*. [Furukawa & Ponce, 2010]

Not intended as an exhaustive literature review. Check the bibliography for more information ...

- Bibliography:

S. M. Seitz, B. Curless, J. Diebel, D. Scharstein and R. Szeliski. "A Comparison and Evaluation of Multi-View Stereo Reconstruction Algorithms," *2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06)*, New York, NY, USA, 2006, pp. 519-528, doi: [10.1109/CVPR.2006.19](https://doi.org/10.1109/CVPR.2006.19).

Multi-View Stereo (MVS). Limitations & MVS-DL approaches

- Produces dense point clouds or meshes from input images
 - Much denser than sparse SfM, thus they can complement each other.
 - Basis for classical photorealistic rendering.
- Limitations:
 - Struggles with textureless, reflective, or transparent surfaces.
 - Requires many images and good calibration.
 - Computationally expensive.
- Modern Learning-Based MVS
 - Recent methods repurpose handcrafted photometric consistency with neural networks:
 - MVSNet (Yao et al., 2018) → deep plane-sweep stereo with cost volume regularization.
 - *Recurrent MVSNet for High-Resolution Multi-View Stereo Depth Inference, 2019*
 - CasMVSNet, R-MVSNet, PatchMatchNet
 - Refinements with hierarchical or patch-based matching.
 - ...
 - Learning methods for the multi-view scenario remain an open challenge in the case of large-scale scenes with high-resolution images, as they have limited scalability due to the high computational costs.
- Bibliography:

Wang et al. "Multi-view stereo in the Deep Learning Era: A comprehensive review", *Displays, Volume 70, 2021, 102102, ISSN 0141-9382*, <https://doi.org/10.1016/j.displa.2021.102102>.

Stathopoulou and Remondino. "A survey on conventional and learning-based methods for multi-view stereo." *The Photogrammetric Record* 38.183 (2023): 374-407.

Green AI in Multi-View Stereo (MVS)

-  Classical MVS: **No training**, only inference → greener at small scale.
- DL-MVS (MVSNet) was not green (huge compute, GPU hours).
 - **Huge training cost**, but inference can be more robust and faster per scene (esp. optimized versions). DL-MVS trades efficiency per scene for generalization (after training).
- Later works emphasize efficiency optimizations:
 - Memory reduction, coarse-to-fine, sparse sampling.
 - Research trend: **efficiency-aware designs** (CasMVSNet, PatchmatchNet, Fast-MVSNet).

Structure from Motion (SfM)

Structure from Motion (SfM). Goal

- Compute 3D scene structure and camera motion from a sequence of images

Structure from Motion (SfM). Goal

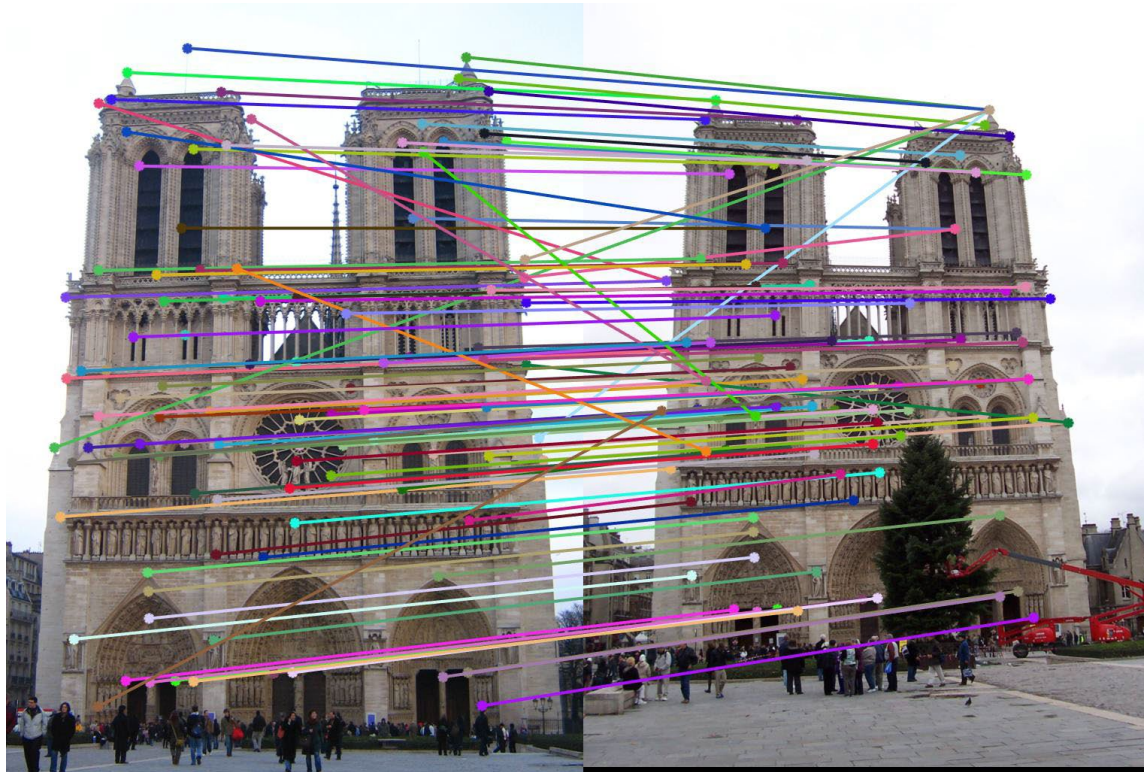
- Compute 3D scene structure and camera motion from a sequence of images
- Detect keypoints (and their feature descriptors).



SIFT descriptors. Source: https://sites.cc.gatech.edu/classes/AY2016/cs4476_fall/results/proj2/html/scalhoun9/index.html

Structure from Motion (SfM). Goal

- Compute 3D scene structure and camera motion from a sequence of images
- Detect keypoints (and their feature descriptors).
 - Track keypoints through descriptor template matching, optical flow, ...



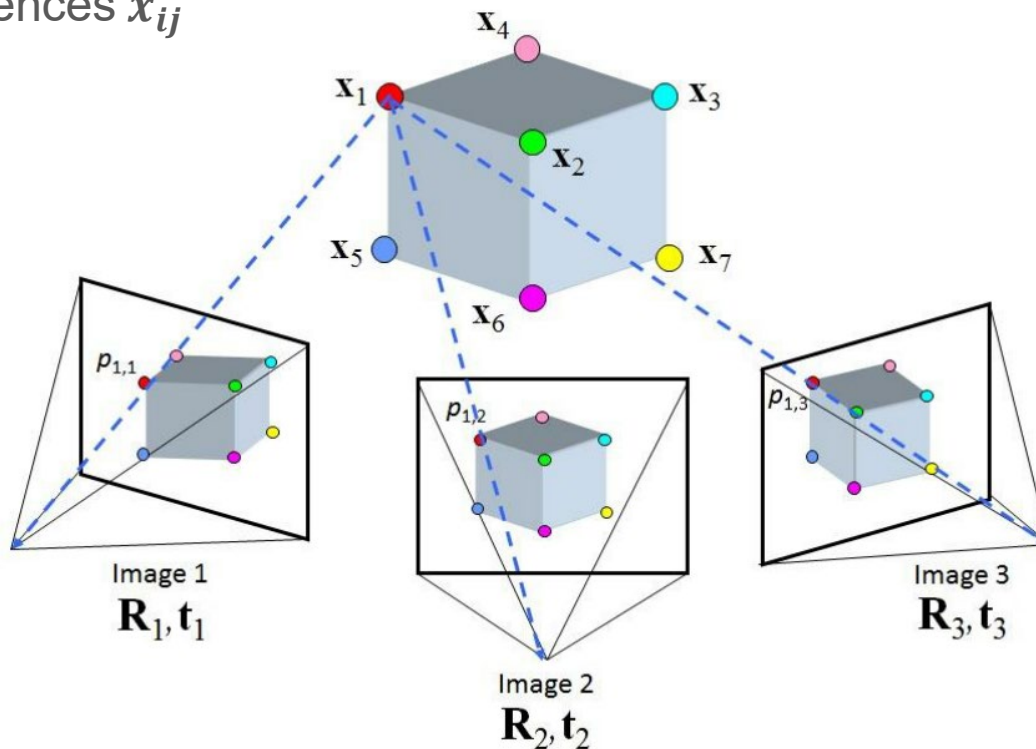
Source: https://sites.cc.gatech.edu/classes/AY2016/cs4476_fall/results/proj2/html/scalhoun9/index.html

Structure from Motion (SfM). Problem Definition

- Given m images of n fixed 3D points

$$x_{ij} = P_i X_j, \quad i = \{1, \dots, m\}, \quad j = \{1, \dots, n\}$$

- Problem:** estimate m projection matrices P_i and n 3D points X_j from the mn correspondences x_{ij}



Source: Yilmaz and Karakus. Stereo and kinect fusion for continuous 3D reconstruction and visual odometry

Structure from Motion (SfM). Key idea (Tomasi & Kanade)

- Assume P_i are orthographic projections
- High level idea:
 - Compute essential/fundamental matrix between image pairs. Use normalized frame coords.
 - Decompose to get relative rotation (R) and translation (t) between cameras.

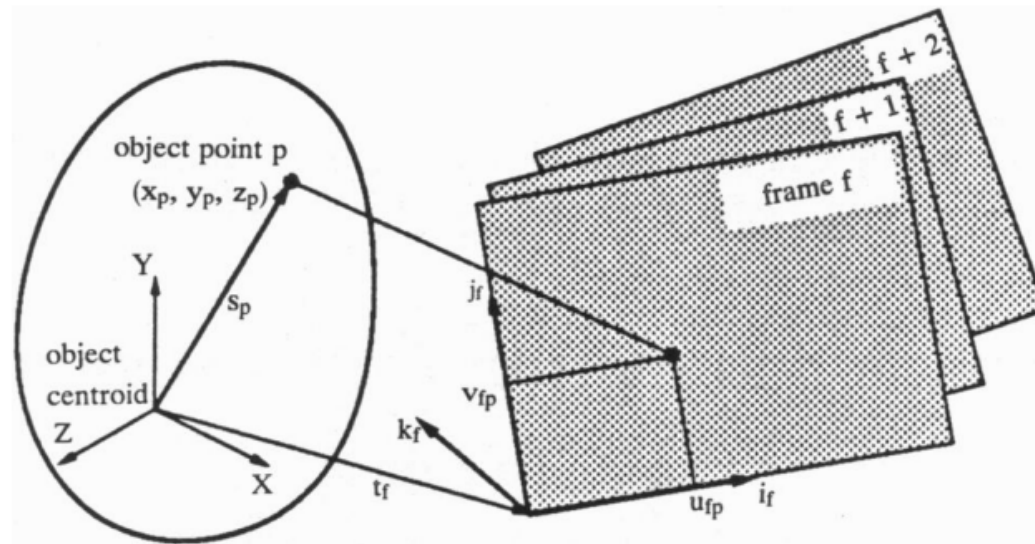


FIG. 1. Systems of reference used in our problem formulation.

Structure from Motion (SfM). Key idea (Tomasi & Kanade)

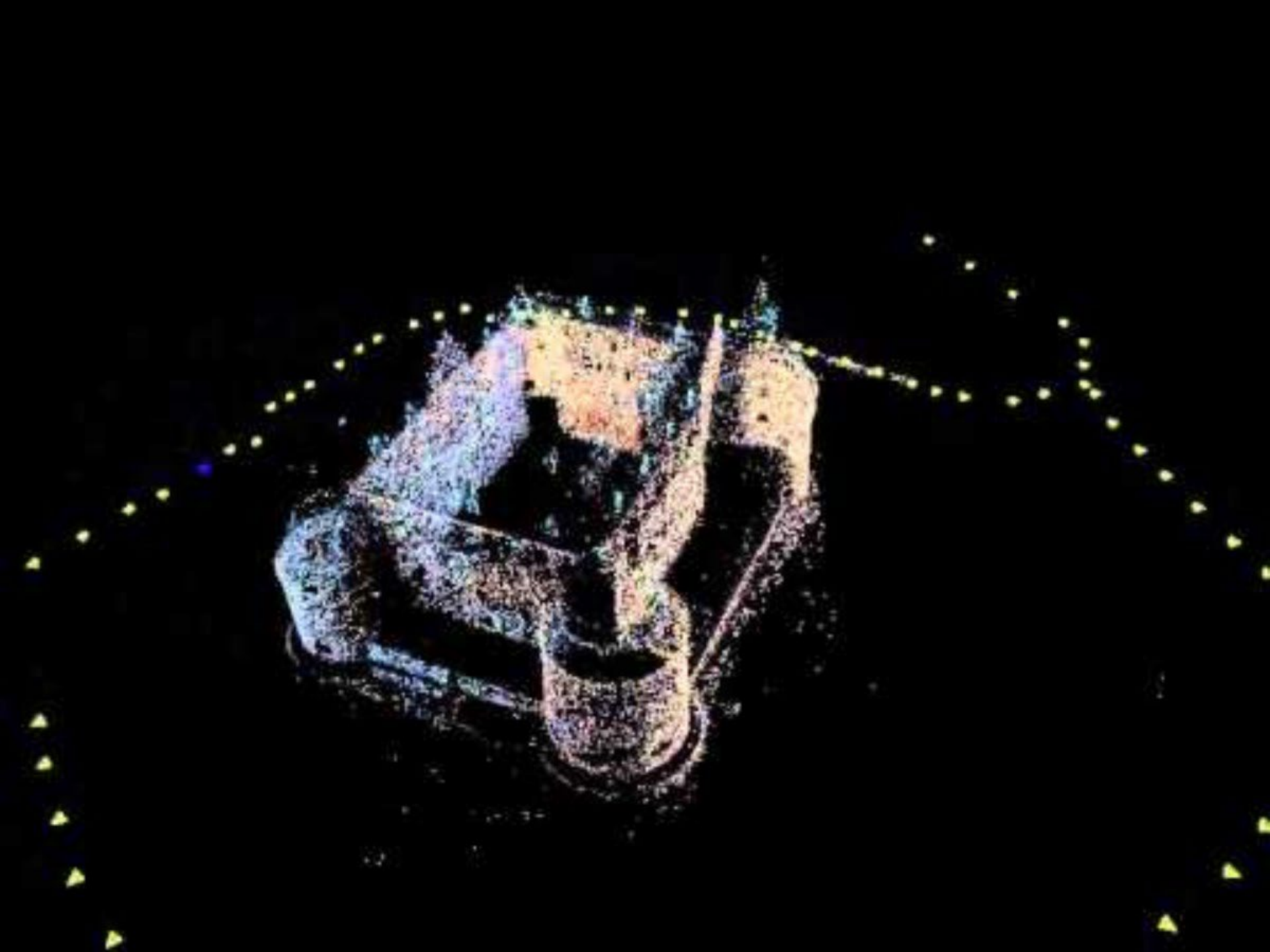
- Assume $\mathbf{P}_i$ are orthographic projections
- Key idea:
 1. Detect and track feature points.
 2. Create the centroid subtracted matrix $\mathbf{W}$ of corresponding feature points.
 3. Compute SVD of $\mathbf{W}$ and enforce rank constraint.

$$\mathbf{W} = \mathbf{U} \Sigma \mathbf{V}^T = \mathbf{U}_1 \Sigma_1 \mathbf{V}_1^T \quad \text{economic representation}$$

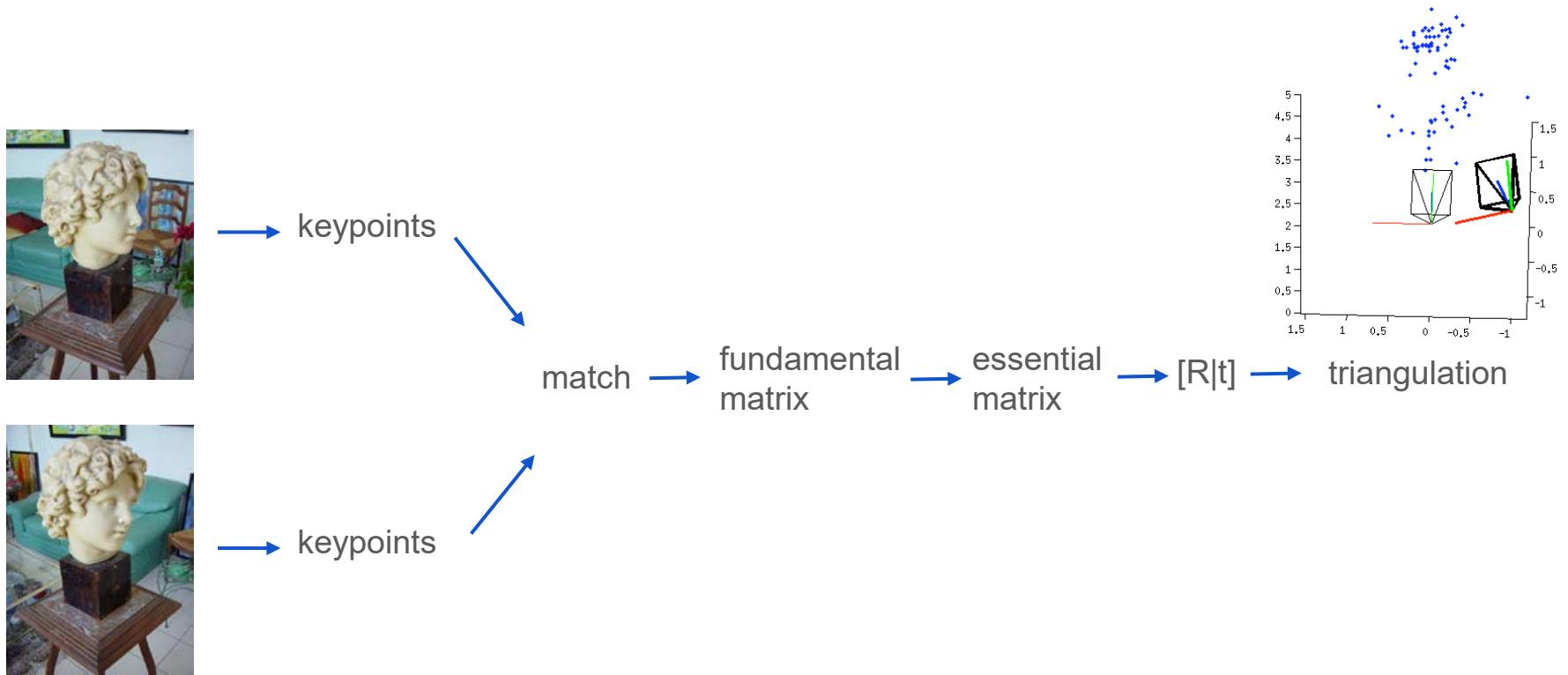
(2Fx3)(3x3)(3xP)

4. Set $\mathbf{M} = \mathbf{U}_1 (\Sigma_1)^{1/2} \mathbf{Q}$ (motion) and $\mathbf{S} = \mathbf{Q}^{-1} (\Sigma_1)^{1/2} \mathbf{V}_1^T$ (structure)
5. Find $\mathbf{Q}$ (3x3) by enforcing the orthonormality constraint.

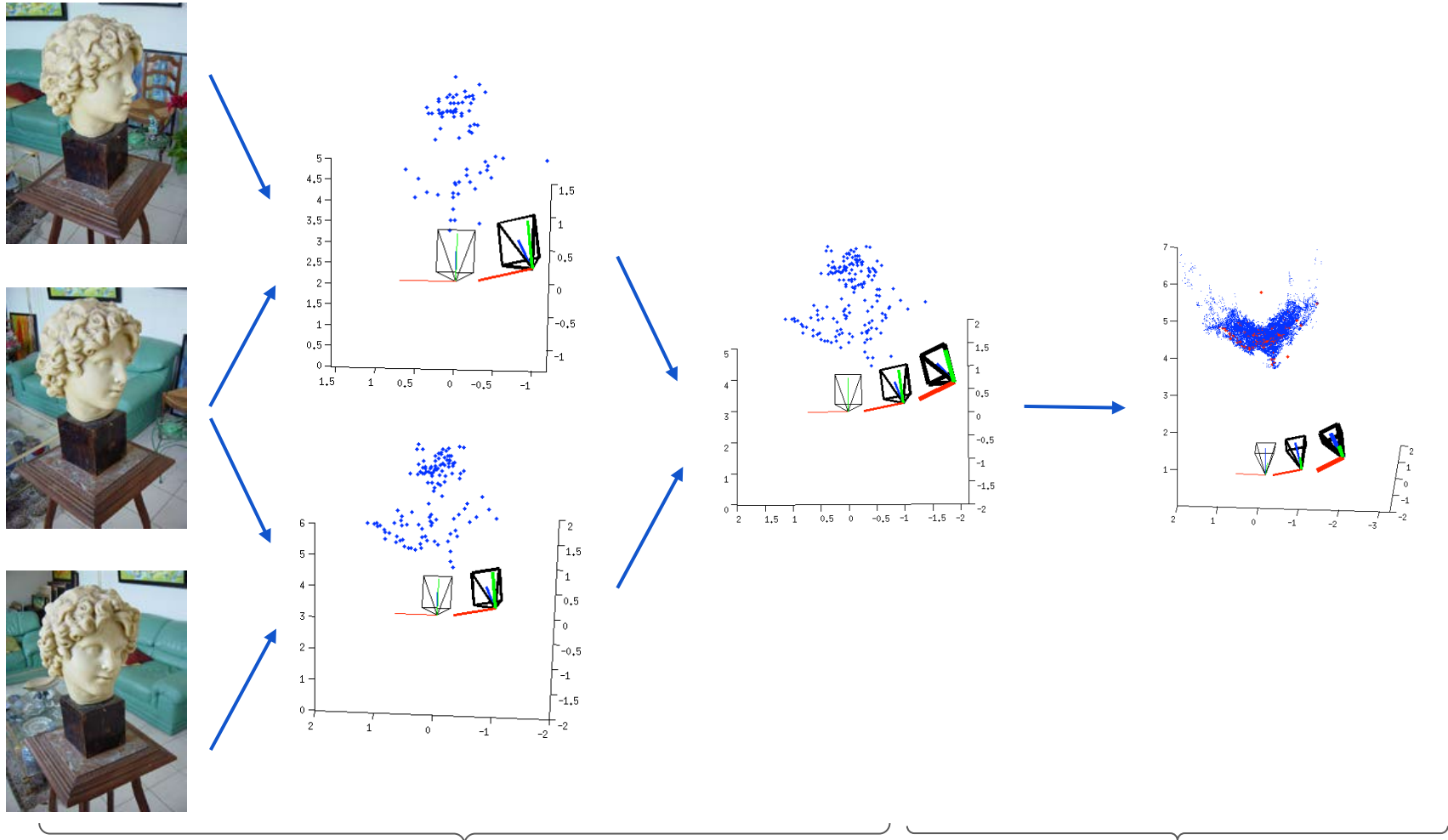
Source: Shree K. Nayar (Columbia University) <https://www.youtube.com/watch?v=0BVZDyRrYtQ>



From Stereo Reconstruction Pipeline



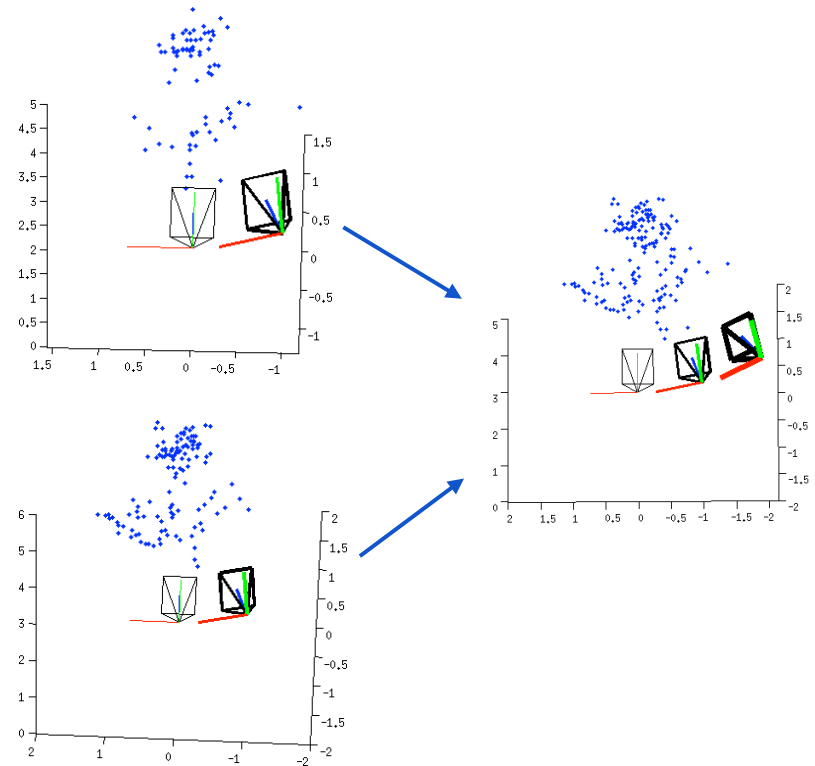
Incremental SfM + MVS Reconstruction Pipeline



Incremental SfM & RANSAC (Random Sample Consensus)

- Choose one image as **reference / origin**.
- Initialize its pose $R_0 = I, t_0 = 0$.
- For each connected image pair:
 1. Recover **relative pose** ($R_{i \rightarrow j}, t_{i \rightarrow j}$) using essential matrix E_{ij} .
 2. Use **RANSAC** on point correspondences to discard outliers.
 3. Chain relative poses to global frame $\rightarrow$ incrementally get (R_j, t_j).
- **Key idea:** Camera intrinsics + pose recovery are repeated for **each new camera**, either using pairwise relative poses or via global **bundle adjustment**.

- Incremental Pose Estimation



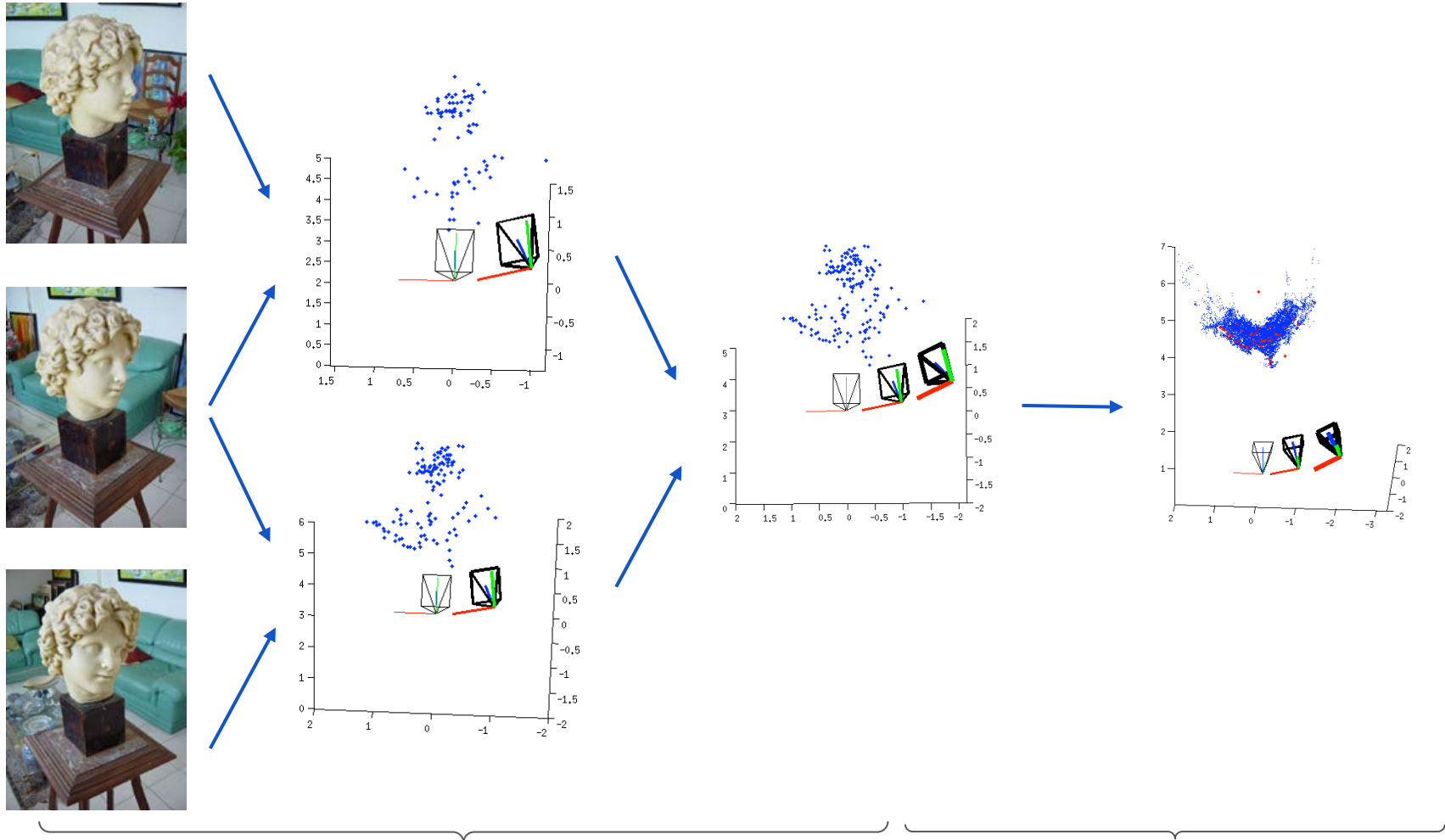
Structure from Motion (SfM). Observations

- No rectification is possible initially because intrinsics/poses are unknown.
- SfM's job is to recover intrinsics + extrinsics
 - → this produces the calibration that later enables MVS.
- After SfM, one could rectify pairs if needed, but most dense MVS algorithms already work in the unrectified, calibrated setting.

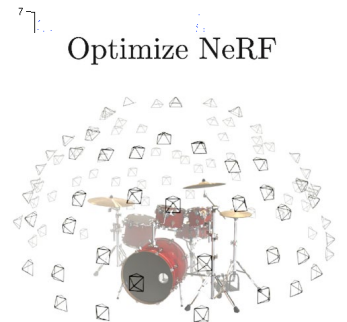
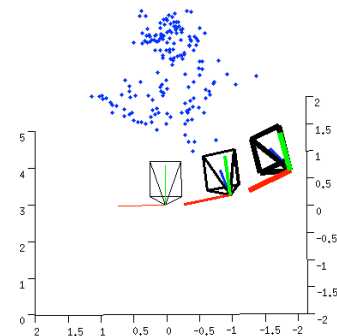
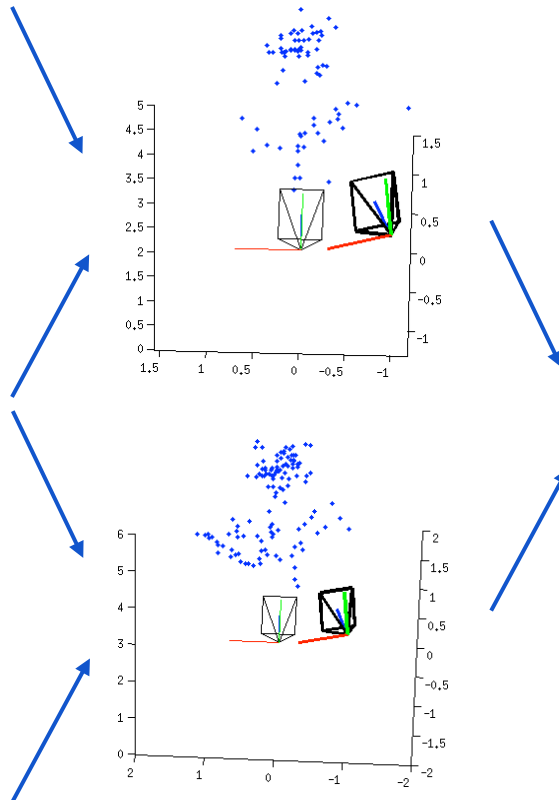
Session Outline. Next

1. Photogrammetry and Computer Vision Fundamentals
- 2. A gentle introduction to Novel View Synthesis (NVS)**
3. Synthetic image data generation for AI consumption

Incremental SfM + MVS Reconstruction Pipeline



Incremental SfM + MVS Reconstruction Pipeline



Optimize NeRF

Train a Neural Radiance Field!

Structure from Motion (SfM)

Multi-view Stereo (MVS)

INDITEX UDC

Cátedra de IA en Algoritmos Verdes

GRACIAS POR SU ATENCIÓN

<https://catedrainditexalgoritmosverdes.com/>

MICROCREDENCIAL ALGORITMOS VERDES PARA LA IA: DISEÑO E IMPLEMENTACIÓN

Tema: AI, Graphics & Visual Computing: Usos de la IA para la
Reconstrucción 3D. Generación de Datos Sintéticos.

Sesión: Lunes 6 de octubre de 2025

Ponente: José A. Iglesias Guitián



UNIVERSIDADE DA CORUÑA



Financiado por la
Unión Europea
NextGenerationEU



GOBIERNO
DE ESPAÑA
MINISTERIO
PARA LA TRANSFORMACION DIGITAL
Y DE LA FUNCIÓN PÚBLICA



España | digital

20
26